



Complete Technical Specification

Version 1.1.0 | Copyright © 2026 Void Engine | Licensed under the MIT License

Foreword: The Philosophy of Void

The Problem With Big Engines

Engines like Unity and Unreal try to do everything. They give you physics, networking, UI, animation, visual scripting, and everything else you can think of.

The problem is your game is unique.

Take physics. A 2D platformer needs simple collision detection and gravity. A racing game needs vehicle physics with traction, weight transfer, and suspension. A puzzle game might need no physics at all. Yet Unity and Unreal force their physics systems on you whether you need them or not. You can't strip out what you don't need. You can't replace it with something better suited for your game.

UI is the same story. Unity uses UGUI. Unreal uses UMG. You use what they give you or you fight the system to build something custom. This is why many Unity games use third-party UI solutions like Noesis or Coherent. They are fighting the built-in system because it doesn't fit their needs.

Rendering follows the same pattern. Unity's built-in render pipeline was slow and inflexible for years. They eventually created Scriptable Render Pipelines (URP and HDRP) to fix this. But these came years late, and many developers had already built workarounds or switched to other engines. Unreal's rendering is powerful but famously complex. Want to understand how it works? You'll need to read through hundreds of thousands of lines of C++ code. Want to change something? Good luck.

Networking? Unity has UNET (deprecated), then Mirror, then Netcode for GameObjects. Unreal has its own replication system. Each one forces a specific architecture on your game. Want to use a different networking library? You're fighting the engine.

Animation systems are no different. Unity has Mecanim. Unreal has Animation Blueprints. Both are powerful. Both are also rigid. Want to implement a custom animation system? You're fighting the engine.

And asset pipelines? Unity has AssetBundles and Addressables. Unreal has PAK files. Both are complex. Both have quirks. Both leave your assets exposed to extraction tools. Want to encrypt your assets? You're fighting the engine.

This is the hidden cost of big engines. You don't just pay with money. You pay with time fighting the engine. You pay with performance carrying systems you don't need. You pay with frustration when something doesn't work the way you expect.

You pay with the freedom to do things your way.

You fight the engine instead of making your game.

The Problem With Bare Frameworks

At the other extreme, frameworks like MonoGame and FNA give you almost nothing. They handle rendering, windowing, and input. That's it.

The problem is these systems are too basic. They give you the raw tools but none of the supporting infrastructure that every game needs.

Take rendering. MonoGame and FNA let you draw sprites. That's it. If you want texture atlasing to reduce draw calls and improve performance, you have to build it yourself. You have to write your own packing algorithm, your own atlas management, your own defragmentation system. You have to handle texture page allocation, LRU eviction, and GPU uploads. This is a significant amount of work that every game benefits from. Yet every developer has to rebuild it.

Asset loading is the same story. These frameworks give you file I/O. That's it. If you want automatic asset caching so textures and sounds load faster the second time you use them, you build it yourself. If you want LRU eviction to manage memory, you build it yourself. If you want mount-based virtual file systems so you can load assets from different sources, you build it yourself. If you want encrypted asset packs to protect your work from extraction tools, you build that too. Every game needs these things. Every developer builds them from scratch.

Audio follows the same pattern. They give you the ability to play sounds. That's it. If you want pooling to avoid allocating new objects every time a sound plays, you build it yourself. If you want priority-based voice stealing so important sounds play over ambient ones, you build it yourself. If you want category volumes so you can control music, SFX, and UI sounds separately, you build it yourself. If you want background updates so sounds don't block the main thread, you build it yourself. Every game needs these things. Every developer rebuilds them.

Saving and loading? They give you file streams. That's it. If you want encryption to prevent players from cheating, you build it yourself. If you want versioning so save files from older versions still work, you build it yourself. If you want integrity verification to detect corrupted saves, you build it yourself. If you want atomic writes so a crash doesn't corrupt the save file, you build it yourself. Every game needs saving. Every developer rebuilds the same system.

Pathfinding? Nothing. If you want enemies that can navigate around obstacles, you build it yourself. You write your own A* implementation, your own node graph, your own heuristic functions. A* is a solved problem. It has been implemented thousands of times. Why do it again?

Coroutines? Nothing. If you want to write code that runs over multiple frames, you build it yourself. You write your own coroutine manager, your own yield instructions, your own wait system. Every game needs timers, animations, and sequences. Every developer writes the same coroutine system.

Logging? They give you `Console.WriteLine`. That's it. If you want log levels so you can filter messages by severity, you build it yourself. If you want categories so you can see which system logged the

message, you build it yourself. If you want file rotation so logs don't fill up the disk, you build it yourself. If you want async writing so logging doesn't block the game, you build it yourself. Every game needs logging. Every developer builds the same logger.

The same goes for input handling beyond the basics, state machines, animation systems, UI frameworks, and dozens of other systems. These are all solved problems. They have been solved thousands of times by thousands of developers. Yet bare frameworks force you to solve them again. Every project. Every time.

This is the hidden cost of bare frameworks. You spend your time rebuilding the same systems over and over instead of making your game. You waste time reinventing wheels. You delay shipping your game because you're still writing code that has already been written a thousand times before.

You build infrastructure instead of making your game.

The Void Solution

Void sits in the middle.

It gives you the essential building blocks every game needs: rendering, assets, input, audio, saving, pathfinding, coroutines, and logging. It then gets out of your way.

Unlike big engines, Void doesn't force you into rigid systems. You aren't locked into a specific physics engine or UI framework. You bring what you need and leave what you don't. You aren't forced to carry around physics code when your game is a puzzle game. You aren't forced to use a UI system that doesn't fit your vision. You aren't forced to use a render pipeline that doesn't suit your art style. You choose what you need and build the rest yourself. The engine doesn't fight you. It works with you.

Unlike bare frameworks, Void doesn't make you rebuild solved problems. The essentials are already there, tested and working. You don't have to write your own texture atlasing system. You don't have to write your own audio pool with priority-based voice stealing. You don't have to write your own asset cache with LRU eviction. You don't have to write your own encrypted save system. You don't have to write your own A* pathfinding. You don't have to write your own coroutine manager. These systems are already built, already tested, already working. You just use them.

This balance is what makes Void different. You get the foundation without the bloat. You get the freedom without the friction.

You get rendering. Not just drawing sprites, but efficient batch rendering with automatic texture atlasing, shader support, and post-processing effects. You can draw thousands of sprites with minimal draw calls. You can apply bloom, blur, and color grading to your entire scene.

You get assets. Not just file loading, but automatic caching, LRU eviction, and mount-based virtual file systems. You can load assets from disk, from encrypted packs, or from custom sources. You never load the same asset twice. You never worry about memory bloat.

You get input. Not just reading keys and buttons, but action-based input that separates what the player does from how they do it. You define actions like "Jump" and "MoveLeft" and bind them to keyboard keys, mouse buttons, and gamepad buttons. The player can remap controls easily. Your code stays clean and readable.

You get audio. Not just playing sounds, but pooled sound instances, priority-based voice stealing, category volumes, and background updates. You can play hundreds of sounds without allocating new objects. You can control music, SFX, and UI sounds separately. You never block the main thread with audio processing.

You get saving. Not just writing files, but encrypted save data with manifest verification, version checking, and atomic writes. Players can't cheat by editing save files. Corrupted saves are detected. Save files from older versions are handled gracefully.

You get pathfinding. Not just a blank slate, but A*, Dijkstra, BFS, and Greedy algorithms with multiple heuristic functions and diagonal movement modes. You also get flow fields for hundreds of agents moving toward the same target. Your enemies can navigate your levels without you writing a single line of pathfinding code.

You get coroutines. Not just timers, but a complete coroutine system with tweens, sequences, concurrent execution, and conditional waits. You can write animations that run over time. You can sequence actions one after another. You can wait for conditions to become true. Your code stays clean and readable.

You get logging. Not just `Console.WriteLine`, but asynchronous multi-sink logging with log levels, categories, file rotation, and colored console output. You can log to files, to the console, or to custom sinks. You never block the game with logging. You can filter messages by severity. You can categorize logs by system.

And you get all of this without the bloat. Without the rigid systems. Without the forced architectures. Without the hidden costs.

You get the essentials. Then you build the rest.

This balance is what makes Void different. You get the foundation without the bloat. You get the freedom without the friction.

You make your game instead of fighting an engine or rebuilding infrastructure.

Extensibility Over Lock-In

Void doesn't tell you how to make your game. It gives you the tools and steps aside.

The engine is built around interfaces and base classes, not rigid systems. You can swap out any part without fighting the framework. This keeps your code clean, your options open, and your game your own.

Think about what that means in practice. In Unity, if you don't like the way `AssetBundles` work, you're stuck. You can't replace the asset loading system. You can't add encryption. You can't change how assets are packed. You can try to work around it, but you're fighting the engine. In Void, you just implement `IMount` and plug it in. The engine doesn't care where your assets come from. It just asks the mount for files.

In Unreal, if you don't like the way PAK files work, you're stuck. You can't change the format. You can't add your own compression. You can't use a different encryption scheme. You can try to modify the engine source, but then you're maintaining a fork. Good luck merging updates. In Void, you just implement `IMount`. The engine doesn't care about the format. It just asks the mount for data.

In MonoGame, if you want texture atlasing, you build it yourself. You write your own packing algorithm, your own atlas management, your own defragmentation system. You're rebuilding something that every game needs. In Void, you can use the built-in `SkylinePacker` or `GuillotinePacker`. Or you can implement `IAtlasPacker` and plug in your own algorithm. The engine doesn't care how you pack textures. It just asks the packer for rectangles.

In any engine with a built-in save system, you're locked into their format, their encryption, their versioning scheme. You can't change it. You can't add your own integrity checks. You can't use a different serialization format. In Void, you inherit from `ContentTypeWriterReader` and implement your own `Write` and `Read` methods. You control exactly what gets saved and how. The engine handles the encryption, the versioning, the atomic writes, and the integrity verification. You handle your data.

Here is how that works in practice:

Every major system in Void exposes an interface. Want a different texture packing algorithm? Implement `IAtlasPacker` and plug it in. Need assets from a network drive or cloud storage? Implement `IMount` and add it to the system. Want logs sent to a database instead of a file? Implement `ILogSink` and register it. Need to save custom data types? Inherit from `ContentTypeWriterReader` and the save system handles the rest.

Want a custom blend mode for a special visual effect? Implement `IBlendMode` and use it in your batcher. Need a custom render target for off-screen rendering? Implement `IRenderTarget` and plug it in. Want a custom vertex buffer for efficient particle rendering? Implement `IVertexBuffer` and use it in your batcher. Need a custom shader for a specific visual effect? Implement `IShader` and use it in your batcher.

You don't modify the engine code. You don't fork the repository. You don't write a single line of engine code. You just implement the interface and register it. The engine finds it and uses it. Automatically.

This is the power of extensibility over lock-in. You get the foundation without the bloat. You get the freedom without the friction. You can change anything without fighting the engine. You can add anything without rebuilding the foundation.

The engine doesn't care how you do it. It just asks for the result. That's the beauty of interfaces. The engine defines what it needs. You provide the implementation. The engine uses it. Nothing is hardcoded. Nothing is locked in. Nothing is forced upon you.

This keeps your code clean, your options open, and your game your own.

You make your game instead of fighting an engine. You extend instead of rebuilding. You own your code instead of borrowing it.

The Physiology of Void

Beyond the code and systems, Void has a philosophy. It's about how we handle the things that aren't there. Missing data. Destroyed objects. Null references. Failed operations.

This philosophy is called the Physiology of Void.

Explicit over implicit. No silent nulls. If something can be missing, we make it obvious. We use nullable types explicitly. We don't hide failures. We surface them so you can handle them.

NonNull by default. Reference types are assumed to be non-null unless explicitly marked otherwise. This catches bugs early. If something should never be null, we enforce it.

Validation at boundaries. Public APIs validate their inputs. Internal code can assume valid state. This means bugs are caught at the edges, not deep inside the engine where they're harder to debug.

Try patterns over exceptions. Operations that can fail have Try versions. TryLoad, TrySave, TryGet. These return a boolean and an out parameter. You check the boolean and handle the failure gracefully. No try/catch blocks cluttering your code.

Dispose patterns for resources. Every resource that needs cleanup implements IDisposable. Sounds, textures, render targets, vertex buffers. You dispose them when you're done. The engine handles the rest.

Pooling over allocation. Sound instances, render targets, vertex buffers. These are pooled and reused. Less garbage collection. Less stuttering. Better performance.

Weak references for caches. When caching is optional, we use weak references. The cache doesn't prevent garbage collection. Memory is freed when it's needed.

The Physiology of Void is about being honest about what's there and what isn't. It's about making the safe path the easy path. It's about catching bugs early and handling failures gracefully.

This philosophy is baked into every system in Void. It's not an afterthought. It's how the engine was designed from the start.

Why Asset Protection Matters

When you release a game, your assets are your intellectual property. Art, music, levels, dialogue. These represent money, time, and creative work.

Here is how the problem works in practice:

1. Someone buys your game on a platform like Steam or Itch.io
2. They use readily available extraction tools to open your asset files
3. They pull out every image, sound file, and level
4. They list your entire asset library on asset stores as "Royalty Free" or "Commercial Use"
5. Other developers buy these assets thinking they are legitimate
6. Those developers use your assets in their own games and release them
7. You discover your work being sold and used without your permission
8. Those developers get DMCA takedown notices because they unknowingly used stolen work
9. Everyone loses time, money, and reputation

This is not hypothetical. It happens constantly. Most engines leave your assets exposed by default. Unity asset bundles can be opened with free tools. Unreal PAK files are well documented. MonoGame and FNA leave everything as loose files on disk.

Void takes a different approach. Asset protection is built into the foundation.

Assets are packed into encrypted, tamper-proof archives using AES-GCM 256-bit encryption. The same standard used by governments and militaries. Without the key, the pack is just random bytes. Extraction tools do not work. Your work stays yours.

The pack format is designed to be secure by default. The header is encrypted separately from the data. Each file has a CRC32 checksum for integrity verification. The entire pack is authenticated, so tampering is detected. This isn't security theater. It's real protection.

Who Void Is For

Void is for developers who want control without the chaos.

It's for indie developers who want a lightweight foundation without rebuilding everything. It's for teams that need asset protection built in. It's for anyone tired of fighting their engine. It's for developers who want to make games, not engines.

Void is lightweight enough for small projects and robust enough for large-scale development. The extensibility and asset protection systems scale to any size team or project.

Glossary

Section	Page
1. System Overview	1
2. Getting Started	2
3. Game Lifecycle	3
4. Math Library	4
5. Rendering	5
6. Audio	6
7. Input	7
8. Asset Management	8
9. Asset Packer	9
10. Asset Packer CLI	10
11. Save/Load System	11
12. Pathfinding	12
13. Coroutines	13
14. State Machine	14
15. Beacons (Pub/Sub)	15
16. Logging	16
17. Helpers	17
18. Extensions	18
19. Performance	19
20. Content Pipeline	20
21. Extending Void	21
22. Debugging & Development Tools	22
23. FAQ	23
Appendix A: Platform Support	A
Appendix B: Dependencies	B
Appendix C: License	C
Appendix D: File Manifest	D

1. System Overview

What is Void and why does it exist?

What Is Void?

Void is a lightweight, extensible 2D game engine built for developers who want control without rebuilding everything from scratch.

It sits in the middle of two extremes:

Big engines like Unity and Unreal try to do everything. They give you physics, networking, UI, animation, visual scripting, and more. The problem is they force their systems on you. You can't strip out what you don't need. You can't replace what you don't like. You fight the engine instead of making your game.

Bare frameworks like MonoGame and FNA give you almost nothing. They handle rendering, windowing, and input. That's it. Everything else—texture atlasing, asset caching, audio pooling, pathfinding, saving, logging—you have to build yourself. Every time. For every project. You rebuild infrastructure instead of making your game.

Void gives you the essential building blocks every game needs. It then gets out of your way. You get the foundation without the bloat. You get the freedom without the friction.

What Void Gives You

System	What It Does	Why You Need It
Rendering	Draws sprites, text, shapes, and effects. Uses batch rendering and automatic texture atlasing for performance.	Without it, you'd be writing raw graphics code. With it, you just call <code>Draw()</code> .
Audio	Plays sounds with pooling, priority-based voice stealing, and category volumes.	Without it, you'd manage sound buffers and voice allocation yourself. With it, you just call <code>Play()</code> .
Input	Reads keyboard, mouse, and gamepad. Supports action mapping so you name inputs instead of hardcoding keys.	Without it, you'd read raw device input and track state yourself. With it, you just check <code>IsPressed("Jump")</code> .
Assets	Loads, caches, and manages textures, sounds, fonts, and levels. Supports LRU eviction and virtual file system mounts.	Without it, you'd load files from disk manually and manage memory yourself. With it, you just call <code>Load<Texture>("player.png")</code> .
Asset Packer	Encrypts and compresses your assets into tamper-proof archives using AES-GCM	Without it, your assets are exposed to extraction tools. With it, your work

System	What It Does	Why You Need It
	256-bit encryption.	stays yours.
Save/Load	Saves and loads game data with encryption, versioning, and integrity verification.	Without it, you'd write raw files that players can edit. With it, saves are secure and tamper-proof.
Pathfinding	Finds paths from A to B using A*, Dijkstra, BFS, and Greedy algorithms. Includes flow fields for hundreds of agents.	Without it, you'd write your own pathfinding algorithm. With it, enemies can navigate your levels automatically.
Coroutines	Writes code that runs over multiple frames. Supports delays, tweens, sequences, and conditional waits.	Without it, you'd manage timers and state machines manually. With it, you write sequential code that pauses and resumes.
State Machines	Manages game states (Idle, Walk, Jump, etc.) with coroutine-based states and history support.	Without it, you'd use messy if/else chains. With it, states are clean and transitions are automatic.
Beacons	Decoupled communication between systems using publish/subscribe.	Without it, systems need direct references to each other. With it, they communicate without knowing who's listening.
Logging	Records what your game is doing with levels, categories, file rotation, and async writing.	Without it, you'd use Console.WriteLine and lose messages. With it, you have a record of what happened.

Who Void Is For

Indie developers who want a lightweight foundation without rebuilding everything.

Small teams that need asset protection built in.

Anyone tired of fighting their engine and wants control without the chaos.

Developers who want to make games, not engines.

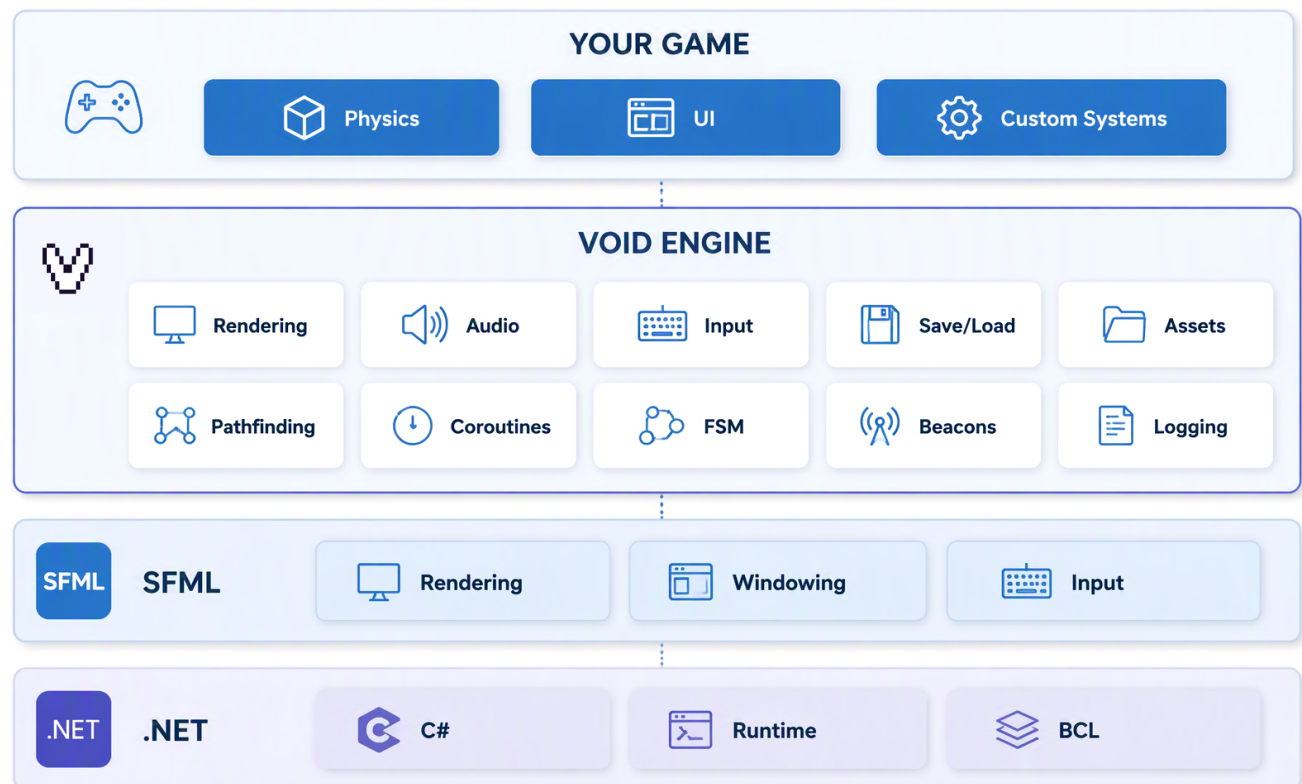
What Void Is Not

Void is a focused 2D engine. It doesn't try to be everything to everyone.

Feature	Status	Why
Physics	Not included	Every game needs different physics. A platformer needs simple collision. A racing game needs vehicle physics. A puzzle game needs none. You bring your own or use the collision helpers.
UI Framework	Not included	Every game has different UI needs. You build your own or use a third-party library.
3D Rendering	Not included	Void is a 2D engine. If you need 3D, look elsewhere.
ECS	Not included	Entity-Component-System is game-specific. You build your own architecture.
Networking	Not included	Every game has different networking needs. You bring your own.

These are intentional omissions. Void gives you the foundation and steps aside. You build what makes your game unique.

How It All Fits Together



Your Game sits on top. This is where you put your physics, UI, and custom logic. Void doesn't care how you structure your game. It just provides the foundation.

Void Engine is the middle layer. This is where all the core systems live. You use them directly. You extend them if needed. You never modify them.

SFML is the graphics, windowing, and input library. Void handles all the SFML interaction. You never touch it directly.

.NET is the runtime. This is what executes your C# code. Void targets .NET 8 and above.

Why This Architecture Works

Separation of concerns. Your game logic is separate from engine systems. Engine systems are separate from the graphics library. Each layer does one thing and does it well.

Extensibility. Every Void system exposes interfaces. You can swap out any part without modifying the engine code. Your changes stay in your code, not in the engine.

Performance. Void is designed to be fast. Batch rendering, texture atlasing, audio pooling, and LRU asset eviction are built in. You don't have to optimize the basics.

Asset protection. AES-GCM 256-bit encryption is built into the foundation. Your assets are protected by default.

Simplicity. Void doesn't try to do everything. It gives you the essentials and gets out of your way. You spend your time making your game, not fighting an engine or rebuilding infrastructure.

The Void Promise

You get the foundation without the bloat.

You get the freedom without the friction.

You make your game instead of fighting an engine or rebuilding infrastructure.

That's what Void is. That's what it does. That's why it exists.

Next: [2. Getting Started](#)

2. Getting Started

Your first Void project in 5 minutes.

What Is This Section?

This section walks you through setting up your first Void project from scratch. You'll learn how to install the engine, create a project, understand the folder structure, and write your first game code. By the end of this section, you'll have a working game window with a player sprite you can move around with your keyboard.

If you've never used a game engine before, this section is for you. If you're experienced, you can skim through and get to the code.

Prerequisites

Requirement	What It Is
.NET 10 SDK	The runtime and development tools for C#. Download from dotnet.microsoft.com .

That's it. Void handles everything else.

Installation

Install the Project Template

The easiest way to start is with the Void project template:

```
dotnet new install Void.Templates
```

This installs the `voidgame` template that generates a complete, runnable project with everything configured.

If you're not using the template, you can add `Void.Engine` manually:

```
dotnet add package Void.Engine
```

Required Settings

SetAppCompany and SetAppName are **required**. Void uses these to create the folder structure for your game. Your saves, logs, and configuration files live in:

Platform	Location
Windows	%APPDATA%/Company/Game/
macOS	~/Library/Application Support/Company/Game/
Linux	~/.config/Company/Game/

If you're not using application data, Void will create a folder inside your bin folder to keep your game's data organized.

Content Folder

Void requires a **Content** folder in your project root. This is where your assets live: images, sounds, fonts, levels, and shaders. The engine enforces this to keep your project clean and organized.

If you're using the template, the Content folder is created automatically. You don't need to do anything.

If you're setting up manually, you'll need to create it:

```
mkdir Content
```

If you prefer a different folder name, use SetContentRoot:

```
.SetContentRoot("MyAssets")
```

Important: Always use a folder name, not a path. The content root should be relative to your application's root directory. Do not use absolute paths or traversal like `../MyAssets`.

Your First Program (Using the Template)

1. Install the template (if you haven't already)

```
dotnet new install Void.Templates
```

2. Create a new game

```
dotnet new voidgame -n MyGame  
cd MyGame
```

Or use the current folder:

```
mkdir MyGame  
cd MyGame  
dotnet new voidgame
```

3. Run your game

```
dotnet run
```

A window will appear. That's it.

Customizing Your Game

When creating a new game, you can specify options:

```
dotnet new voidgame -n MyGame --appCompany MyStudio --appTitle "My Game"
```

Option	Description	Default
-n, --name	The project name	Current folder name
--appCompany	The company name (used for AppData folders)	MyCompany
--appTitle	The display title of the game window	My Game
--TargetFrameworkOverride	Overrides the target framework	net10.0

What You Get

The template generates a complete, runnable game project with:

- `Program.cs` — Entry point with game settings
- `MyGameGame.cs` — Main game class with `OnEnter`, `OnUpdate`, `OnDraw`, `OnExit`
- `Content/` — Folder for your assets
- Pre-configured `.csproj` with `Void.Engine` reference

Manual Setup (Without Template)

If you prefer to set up manually instead of using the template:

1. Create a new console project

```
dotnet new console -n MyGame
cd MyGame
```

2. Add Void.Engine

```
dotnet add package Void.Engine
```

3. Create your game class

Create a new class called `MyGame.cs`:

```
using Void.Engine;

public class MyGame : Game
{
    public MyGame(GameSettings settings) : base(settings) { }

    protected override void OnEnter()
    {
        // Called when the game starts
    }

    protected override void OnUpdate(FrameTime frameTime)
    {
        // Called every frame
    }

    protected override void OnDraw(FrameTime frameTime)
    {
        // Called every frame after OnUpdate
    }

    protected override void OnExit()
    {
        // Called when the game exits
    }
}
```

4. Update Program.cs

```
using Void.Engine;

var settings = GameSettings.Instance
    .SetAppCompany("MyStudio")
    .SetAppName("MyFirstGame")
    .SetWindow(1280, 720)
    .Build();

using var game = new MyGame(settings);
game.Run();
```

5. Run your game

```
dotnet run
```

A window will appear. That's it.

Your First Asset

Let's load a texture and draw it. First, add a texture to your Content folder:

```
Content/
├── Textures/
│   └── player.png
```

Then update your game class:

```
public class MyGame : Game
{
    private Texture _playerTexture;
    private Vect2 _playerPosition = new Vect2(100, 100);

    public MyGame(GameSettings settings) : base(settings) { }

    protected override void OnEnter()
    {
        // Load the texture
        _playerTexture =
AssetManager.Instance.Load<Texture>("Textures/player.png");
    }

    protected override void OnUpdate(FrameTime time)
    {
        // Move the player with arrow keys
        var state = Keyboard.GetState();
        float speed = 200f * time.DeltaTime;

        if (state.IsKeyDown(KeyboardKey.Left)) _playerPosition.X -= speed;
        if (state.IsKeyDown(KeyboardKey.Right)) _playerPosition.X += speed;
        if (state.IsKeyDown(KeyboardKey.Up)) _playerPosition.Y -= speed;
        if (state.IsKeyDown(KeyboardKey.Down)) _playerPosition.Y += speed;
    }
}
```

```

protected override void OnDraw(FrameTime time)
{
    // Draw the player
    var batcher = new SpriteBatcher();
    batcher.Begin(SortMode.BackToFront, BlendMode.Alpha);
    batcher.Draw(_playerTexture, _playerPosition, Color.White);
    batcher.End();
}

protected override void OnExit() { }
}

```

Now when you run your game, you'll see your player sprite on screen. You can move it with the arrow keys.

Common First Steps

Here are some common things you'll want to do next:

Load a Sound

```

Sound explosionSound = AssetManager.Instance.Load<Sound>("Sounds/explosion.wav");
explosionSound.PlayOneShot(volume: 0.8f);

```

Load a Font

```

SpriteFont font = AssetManager.Instance.Load<SpriteFont>("Fonts/arial.png");
batcher.DrawText(font, "Hello World!", new Vect2(10, 10), Color.White);

```

Read Input

```

// Keyboard
var state = Keyboard.GetState();
if (state.IsKeyDown(KeyboardKey.Space)) { /* Jump */ }

// Mouse
var mouse = Mouse.GetState();
if (mouse.IsButtonPressed(MouseButton.Left)) { /* Click */ }

// Gamepad
var gamepad = Gamepad.GetState(PlayerIndex.One);
if (gamepad.IsButtonPressed(GamepadButton.A)) { /* Jump */ }

// Input Actions (recommended)
InputAction.AddAction("Jump").AddKey(KeyboardKey.Space).AddGamepad(GamepadButton.A)
;
var actions = InputAction.GetState();
if (actions.IsPressed("Jump")) { /* Jump */ }

```

Troubleshooting Common Setup Issues

"Content folder not found"

The engine looks for a folder called `Content` in your project root. Make sure it exists and contains your assets.

If you want to use a different folder name:

```
settings.SetContentRoot("MyAssets");
```

"File not found: Textures/player.png"

Check that:

- The file exists in your `Content` folder
- The path is correct (case-sensitive on some platforms)
- The file extension is correct

"No loader found for asset type"

You're trying to load a file type that `Void` doesn't support. Check the supported extensions in the `Asset Management` section.

"Sound pool exhausted"

You're trying to play too many sounds at once. Increase the audio limit in settings:

```
settings.SetAudioLimit(256);
```

Or check that you're disposing sound instances when they finish:

```
instance.SoundCompleted += (s, e) => instance.Dispose();
```

Examples

If you want to see `Void` in action, check out the examples in the `GitHub` repository:

- **FlappyBirb**: A full game using asset packing
- **Scavengers**: A complete roguelite survival game that uses almost every feature of `Void`

What's Next?

Now that you have a basic game running, you can:

- [Learn about the Game Lifecycle](#) — Understand how the game loop works
- [Explore the Math Library](#) — Work with vectors, rectangles, and colors
- [Master Rendering](#) — Draw sprites, text, shapes, and effects
- [Add Audio](#) — Play sounds with pooling and priority control
- [Handle Input](#) — Keyboard, mouse, gamepad, and action mapping
- [Manage Assets](#) — Load, cache, and manage your game assets

Next: [3. Game Lifecycle](#)

3. Game Lifecycle

The Game class, settings, and application flow.

What Is the Game Lifecycle?

The game lifecycle is the sequence of events that happens from the moment your game starts until it exits. Think of it as the heartbeat of your game—a steady rhythm that keeps everything running.

Void manages this heartbeat for you. You don't have to write the loop yourself. You just fill in the methods that matter to your game.

The flow in plain English:

1. **Your game starts** — The window opens, and `OnEnter()` runs once
2. **The game loop begins** — This repeats over and over until you close the window:
 - **Update** — `OnUpdate()` runs. This is where you move things, check inputs, and update game logic
 - **Draw** — `OnDraw()` runs. This is where you draw everything to the screen
3. **Your game exits** — `OnExit()` runs once, and you clean up

This loop runs 60 times per second (or whatever framerate you set). Every cycle, your game updates and draws. This is how games stay responsive and smooth.

The Game Class

Every Void game starts with a class that inherits from `Game`. This is your game's home base.

```
public class MyGame : Game
{
    public MyGame(GameSettings settings) : base(settings) { }

    protected override void OnEnter() { }
    protected override void OnUpdate(FrameTime time) { }
    protected override void OnDraw(FrameTime time) { }
    protected override void OnExit() { }
}
```

You only need to override the methods you actually use. If you don't need `OnEnter` or `OnExit`, you can leave them empty.

The Four Key Methods

OnEnter() — Setup

When: Called once, right after the game starts and the window opens.

What you do here: Load your assets, set up your game state, create your player, enemies, and anything else your game needs to start.

Example:

```
protected override void OnEnter()
{
    // Load assets
    _playerTexture = AssetManager.Instance.Load<Texture>("Textures/player.png");
    _backgroundMusic = AssetManager.Instance.Load<Sound>("Sounds/music.ogg");

    // Set up game state
    _playerPosition = new Vect2(100, 100);
    _playerHealth = 100;
    _score = 0;

    // Start background music looping
    var musicInstance = _backgroundMusic.CreateInstance();
    musicInstance.Looping = true;
    musicInstance.Play();
}
```

Think of this as: Setting up the chessboard before the game starts.

OnUpdate() — Game Logic

When: Called every frame (usually 60 times per second).

What you do here: Move things, check inputs, update game logic, run AI, detect collisions, update timers, and manage game state.

Why it matters: This is where your game actually happens. Everything that changes over time—positions, health, scores, AI decisions—happens here.

Example:

```
protected override void OnUpdate(FrameTime time)
{
    // Move the player with WASD
    var state = Keyboard.GetState();
    float speed = 200f * time.DeltaTime;

    if (state.IsKeyDown(KeyboardKey.W)) _playerPosition.Y -= speed;
    if (state.IsKeyDown(KeyboardKey.S)) _playerPosition.Y += speed;
    if (state.IsKeyDown(KeyboardKey.A)) _playerPosition.X -= speed;
    if (state.IsKeyDown(KeyboardKey.D)) _playerPosition.X += speed;

    // Update enemy AI
    foreach (var enemy in _enemies)
        enemy.Update(time.DeltaTime, _playerPosition);

    // Check collisions
    if (_playerRect.Intersects(_enemyRect))
        _playerHealth -= 10;

    // Check win/lose conditions
    if (_playerHealth <= 0)
        ChangeState(GameState.GameOver);
}
```

Think of this as: The part of the game that runs in the background, constantly updating everything.

OnDraw() — Rendering

When: Called every frame, after `OnUpdate()`.

What you do here: Draw everything to the screen—sprites, text, shapes, UI elements, and effects.

Why it matters: This is what the player actually sees. If `OnUpdate()` is the brain, `OnDraw()` is the eyes.

Example:

```
protected override void OnDraw(FrameTime time)
{
    // Create a batcher for this frame
    var batcher = new SpriteBatcher();

    // Start the batch
    batcher.Begin(SortMode.BackToFront, BlendMode.Alpha);

    // Draw the background
    batcher.Draw(_backgroundTexture, Vect2.Zero, Color.White);

    // Draw the player
    batcher.Draw(_playerTexture, _playerPosition, Color.White);

    // Draw enemies
    foreach (var enemy in _enemies)
        batcher.Draw(enemy.Texture, enemy.Position, enemy.Color);

    // Draw UI
    batcher.DrawText(_font, $"Score: {_score}", new Vect2(10, 10), Color.White);
    batcher.DrawText(_font, $"Health: {_playerHealth}", new Vect2(10, 40),
Color.White);

    // End the batch (sends everything to the GPU)
    batcher.End();
}
```

Think of this as: The artist painting the scene based on the latest state from `OnUpdate()`.

OnExit() — Cleanup

When: Called once, right before the game exits.

What you do here: Save game progress, clean up resources, log final stats, and do any last-minute cleanup.

Example:

```
protected override void OnExit()
{
    // Save player progress
    SaveSystem.Save("player.sav", _playerData);

    // Log final stats
    Logger.Instance.Info($"Game ended with score: {_score}");

    // Stop all sounds
    SoundHelper.StopAll();
}
```

Think of this as: Packing up the chessboard after the game is over.

Understanding FrameTime

FrameTime is passed to `OnUpdate()` and `OnDraw()` every frame. It tells you everything you need to know about timing.

```
protected override void OnUpdate(FrameTime time)
{
    float dt = time.DeltaTime;           // Time since last frame
    float unscaled = time.UnscaledDeltaTime; // Real time (ignores time scale)
    float alpha = time.Alpha;           // For smooth interpolation
    float fps = time.FPS;               // Current frames per second
    bool isFixed = time.IsFixedTimeStep; // Fixed or variable timestep?
}
```

Why DeltaTime Matters

The Problem: If you move something by 5 pixels every frame, it moves at different speeds on different computers. Fast computers run at 60 FPS (300 pixels/second). Slow computers run at 30 FPS (150 pixels/second).

The Solution: Use `DeltaTime` to make movement frame-rate independent.

```
// Wrong: Moves faster on fast computers
_position.X += 5f;

// Correct: Moves the same speed on all computers
_position.X += 200f * time.DeltaTime;
```

What each property does:

Property	What It Tells You	When To Use
<code>DeltaTime</code>	Time since last frame, affected by <code>TimeScale</code>	Moving objects, game logic (most things)
<code>UnscaledDeltaTime</code>	Real time since last frame	UI, audio, debug timers (things that shouldn't slow down)
<code>Alpha</code>	Interpolation value between fixed updates (0-1)	Smooth rendering between fixed updates
<code>FPS</code>	Current frames per second	Debug info, performance monitoring
<code>IsFixedTimeStep</code>	Whether fixed timestep is enabled	Conditional logic
<code>TimeScale</code>	Global speed multiplier (1 = normal)	Slow-motion, fast-forward, pause

Fixed vs Variable Timestep

This is one of the most important concepts in game development. Let me explain it simply.

Variable Timestep (IsFixedTimeStep = false)

How it works: Your game updates once per frame, using the actual time that passed since the last frame.

```
Frame 1: 16ms passed → Update with 16ms  
Frame 2: 18ms passed → Update with 18ms  
Frame 3: 14ms passed → Update with 14ms
```

Pros:

- Simple to implement
- Smooth rendering

Cons:

- Physics becomes inconsistent (jumpy at low FPS)
- Can't reproduce bugs reliably
- Fast computers and slow computers behave differently

When to use: Simple games that don't need precise physics (visual novels, puzzle games, card games).

Fixed Timestep (IsFixedTimeStep = true)

How it works: Your game updates at a fixed interval (like 1/60 of a second), regardless of frame rate.

```
Frame 1: 16ms passed → No update (not enough time)  
Frame 2: 18ms passed → Update with 16ms (one fixed step)  
Frame 3: 14ms passed → No update (not enough time)  
Frame 4: 16ms passed → Update with 16ms (one fixed step)
```

Pros:

- Physics is consistent and predictable
- Same behavior on all computers
- Easier to debug and reproduce bugs

Cons:

- Slightly more complex
- May require interpolation for smooth rendering (Void provides this)

When to use: Most games, especially those with physics or precise timing.

Which Should You Use?

Use fixed timestep for most games. It's the default for a reason. Physics, AI, and game logic all benefit from consistent timing.

Use variable timestep for simple games where physics precision isn't important.

How Void Handles This

Void's `FrameTime.Alpha` gives you the interpolation value between fixed updates. This lets you render smoothly even with fixed timestep.

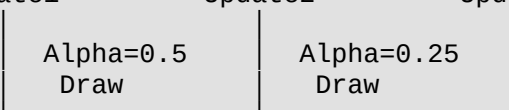
```
protected override void OnDraw(FrameTime time)
{
    // Interpolate between previous and current position for smooth rendering
    float renderX = MathHelper.Lerp(_prevX, _currentX, time.Alpha);
    batcher.Draw(_playerTexture, new Vect2(renderX, 100), Color.White);
}
```

Visual Example

The Alpha value tells you where you are between updates. At Alpha=0, you're at the previous update. At Alpha=1, you're at the next update.

Variable Timestep (no interpolation):
Update -> Draw -> Update -> Draw -> Update -> Draw

Fixed Timestep (with interpolation):
Update1 ————— Update2 ————— Update3



Application Folders

Void creates folders for your game's data automatically. This keeps everything organized.

```
string appFolder = game.ApplicationFolder;    // Root folder
string logsFolder = game.ApplicationLogFolder; // Logs/
string savesFolder = game.ApplicationSaveFolder; // Saves/
string configFolder = game.ApplicationConfigFolder; // Config/
string tempFolder = game.ApplicationTempFolder; // Temp/
```

Where These Folders Live

Platform	Location
Windows	%APPDATA%/Company/Game/
macOS	~/Library/Application Support/Company/Game/
Linux	~/.config/Company/Game/

If you're not using application data, Void creates a folder in your bin directory.

Complete Example

Here's a complete game that shows all the pieces working together:

```
using Void.Engine;

public class MyGame : Game
{
    // Game objects
    private Texture _playerTexture;
    private Texture _enemyTexture;
    private SpriteFont _font;
    private Sound _hitSound;
    private Sound _music;

    // Game state
    private Vect2 _playerPosition;
    private int _health = 100;
    private int _score = 0;
    private float _enemyTimer = 0f;
    private List<Enemy> _enemies = new();

    // Timing
    private Vect2 _prevPlayerPos;
    private Vect2 _currentPlayerPos;

    public MyGame(GameSettings settings) : base(settings) { }

    protected override void OnEnter()
    {
        // Load assets
        _playerTexture =
AssetManager.Instance.Load<Texture>("Textures/player.png");
        _enemyTexture = AssetManager.Instance.Load<Texture>("Textures/enemy.png");
        _font = AssetManager.Instance.Load<SpriteFont>("Fonts/arial.png");
        _hitSound = AssetManager.Instance.Load<Sound>("Sounds/hit.wav");
        _music = AssetManager.Instance.Load<Sound>("Sounds/music.ogg");

        // Start background music
        var musicInstance = _music.CreateInstance();
        musicInstance.Looping = true;
        musicInstance.Play();

        // Set initial position
        _playerPosition = new Vect2(400, 300);
        _prevPlayerPos = _playerPosition;
        _currentPlayerPos = _playerPosition;

        // Spawn some enemies
        for (int i = 0; i < 5; i++)
            SpawnEnemy();
    }

    protected override void OnUpdate(FrameTime time)
    {
        // Save current position for interpolation
    }
}
```

```

    _prevPlayerPos = _currentPlayerPos;

    // Move player
    var state = Keyboard.GetState();
    float speed = 200f * time.DeltaTime;

    if (state.IsKeyDown(KeyboardKey.Left)) _playerPosition.X -= speed;
    if (state.IsKeyDown(KeyboardKey.Right)) _playerPosition.X += speed;
    if (state.IsKeyDown(KeyboardKey.Up)) _playerPosition.Y -= speed;
    if (state.IsKeyDown(KeyboardKey.Down)) _playerPosition.Y += speed;

    _currentPlayerPos = _playerPosition;

    // Update enemies
    foreach (var enemy in _enemies)
        enemy.Update(_playerPosition, time.DeltaTime);

    // Spawn enemies
    _enemyTimer += time.DeltaTime;
    if (_enemyTimer > 2f)
    {
        SpawnEnemy();
        _enemyTimer = 0f;
    }

    // Check collisions
    var playerRect = new Rect2(_playerPosition, _playerTexture.Size);
    for (int i = _enemies.Count - 1; i >= 0; i--)
    {
        if (playerRect.Intersects(_enemies[i].Rect))
        {
            _health -= 10;
            _hitSound.PlayOneShot(0.8f);
            _enemies.RemoveAt(i);

            if (_health <= 0)
                Quit(); // Exit the game
        }
    }
}

protected override void OnDraw(FrameTime time)
{
    var batcher = new SpriteBatcher();
    batcher.Begin(SortMode.BackToFront, BlendMode.Alpha);

    // Draw enemies
    foreach (var enemy in _enemies)
        batcher.Draw(_enemyTexture, enemy.Position, Color.White);

    // Draw player (with interpolation for smooth movement)
    float renderX = MathHelper.Lerp(_prevPlayerPos.X, _currentPlayerPos.X,
time.Alpha);
    float renderY = MathHelper.Lerp(_prevPlayerPos.Y, _currentPlayerPos.Y,
time.Alpha);
    batcher.Draw(_playerTexture, new Vect2(renderX, renderY), Color.White);
}

```

```

        // Draw UI
        batcher.DrawText(_font, $"Health: {_health}", new Vect2(10, 10),
Color.White);
        batcher.DrawText(_font, $"Score: {_score}", new Vect2(10, 40),
Color.White);
        batcher.DrawText(_font, $"Enemies: {_enemies.Count}", new Vect2(10, 70),
Color.White);

        batcher.End();
    }

    protected override void OnExit()
    {
        // Save progress
        Logger.Instance.Info($"Final score: {_score}");
        SoundHelper.StopAll();
    }

    private void SpawnEnemy()
    {
        var position = new Vect2(
            FastRandom.Shared.RangeFloat(0, 800),
            FastRandom.Shared.RangeFloat(0, 600)
        );
        _enemies.Add(new Enemy(position, _enemyTexture));
        _score += 10;
    }
}

// Simple enemy class
public class Enemy
{
    public Vect2 Position { get; private set; }
    public Texture Texture { get; }
    public Rect2 Rect => new(Position, Texture.Size);

    private Vect2 _velocity;
    private float _speed = 50f;

    public Enemy(Vect2 position, Texture texture)
    {
        Position = position;
        Texture = texture;
        _velocity = new Vect2(
            FastRandom.Shared.RangeFloat(-1f, 1f),
            FastRandom.Shared.RangeFloat(-1f, 1f)
        ).Normalized() * _speed;
    }

    public void Update(Vect2 playerPos, float dt)
    {
        // Move toward the player
        var direction = (playerPos - Position).Normalized();
        Position += direction * _speed * dt;
    }
}

```

What's Next?

Now that you understand the game lifecycle, you can:

- [Explore the Math Library](#) — Work with vectors, rectangles, and colors
- [Master Rendering](#) — Draw sprites, text, shapes, and effects
- [Add Audio](#) — Play sounds with pooling and priority control
- [Handle Input](#) — Keyboard, mouse, gamepad, and action mapping

Next: [4. Math Library](#)

4. Math Library

Vectors, rectangles, colors, and utilities for making games.

What Is the Math Library?

Games are all about positions, distances, angles, and colors. The math library gives you the tools to work with these concepts easily.

Without this library, you'd be writing a lot of math code by hand. You'd have to calculate distances, normalize vectors, check if rectangles overlap, and convert colors. This is all code that every game needs. Void provides it so you don't have to write it yourself.

Why You Need This

Imagine making a game without vectors. Every position would be two separate numbers (X and Y). Every time you wanted to move something, you'd have to update both. Every time you wanted to calculate a distance, you'd have to use the Pythagorean theorem. It would be tedious and error-prone.

Vectors turn two numbers into one thing. They make positions, directions, and velocities easy to work with.

Understanding Vect2 (2D Vector)

A `Vect2` represents a point or direction in 2D space. Think of it as an X and Y value that work together.

What you use it for:

- Position of a player, enemy, or object
- Direction and speed of movement
- Distance between two objects

Creating Vectors:

```
// By specifying X and Y
Vect2 position = new Vect2(100, 200);

// Both values the same
Vect2 one = new Vect2(50); // X=50, Y=50

// Pre-defined constants
Vect2 zero = Vect2.Zero;    // (0, 0)
Vect2 up = Vect2.Up;        // (0, -1)
Vect2 right = Vect2.Right;  // (1, 0)
Vect2 down = Vect2.Down;    // (0, 1)
Vect2 left = Vect2.Left;    // (-1, 0)
```

Basic Operations:

```
// Add and subtract
Vect2 a = new Vect2(10, 20);
Vect2 b = new Vect2(5, 10);
Vect2 c = a + b; // (15, 30)
Vect2 d = a - b; // (5, 10)

// Multiply and divide by a number (scaling)
Vect2 scaled = a * 2; // (20, 40)
Vect2 half = a / 2;   // (5, 10)

// Compare vectors
if (a == b) { /* same */ }
if (a != b) { /* different */ }

// Check if a vector is zero
if (a.IsZero) { /* at origin */ }
```

Important Vector Operations:

Operation	What It Does	Example
Length()	Gets the distance from (0,0)	<code>velocity.Length()</code>
LengthSquared()	Gets the squared distance (faster)	<code>velocity.LengthSquared()</code>
Normalized()	Gets a direction vector of length 1	<code>direction.Normalized()</code>
Distance(other)	Distance between two vectors	<code>pos.Distance(target)</code>
DistanceSquared(other)	Squared distance (faster)	<code>pos.DistanceSquared(target)</code>
Dot(other)	Project one vector onto another	<code>a.Dot(b)</code>
Cross(other)	Perpendicular value (2D)	<code>a.Cross(b)</code>
AngleTo(other)	Angle between vectors in radians	<code>a.AngleTo(b)</code>
Lerp(target, t)	Smoothly move from one to another	<code>a.Lerp(b, 0.5f)</code>
Rotate(radians)	Rotate by angle in radians	<code>v.Rotate(PI/2)</code>
Reflect(normal)	Bounce off a surface	<code>v.Reflect(normal)</code>
MoveTowards(target, speed)	Move towards a target	<code>pos.MoveTowards(target, 100)</code>
Clamp(min, max)	Limit values to a range	<code>v.Clamp(new Vect2(0,0), new Vect2(100,100))</code>
Snap(gridSize)	Snap to a grid	<code>v.Snap(32)</code>
Wrap(min, max)	Wrap around a range	<code>v.Wrap(new Vect2(0,0), new Vect2(100,100))</code>

Common Vector Patterns:

Moving toward a target:

```
// Move at a constant speed toward a target
Vect2 direction = (target - position).Normalized();
position += direction * speed * time.DeltaTime;

// Move but stop when you reach the target
position = position.MoveTowards(target, speed * time.DeltaTime);
```

Calculating distance:

```
// Distance between two points
float distance = position.Distance(target);

// Faster if you only need to compare distances
float distSq = position.DistanceSquared(target);
if (distSq < 100 * 100) { /* Within 100 pixels */ }
```

Smooth following (lerp):

```
// Smoothly move toward a target (ease in)
position = position.Lerp(target, 0.1f);
```

Shooting a projectile:

```
// Direction from shooter to target
Vect2 direction = (target - shooterPosition).Normalized();
Vect2 velocity = direction * projectileSpeed;
```

Understanding Rect2 (2D Rectangle)

A Rect2 represents a rectangle with position (X, Y) and size (Width, Height).

What you use it for:

- Collision detection (does this object hit that object?)
- Viewport boundaries (what's visible on screen?)
- UI element placement
- Rendering sprites

Creating Rectangles:

```
// By position and size
Rect2 rect = new Rect2(100, 200, 64, 32);

// From Vect2
Vect2 position = new Vect2(100, 200);
Vect2 size = new Vect2(64, 32);
Rect2 rect = new Rect2(position, size);

// Empty rectangle
Rect2 empty = Rect2.Empty;
```

Properties:

```
Rect2 rect = new Rect2(100, 200, 64, 32);

float x = rect.X;           // 100 (left edge)
float y = rect.Y;           // 200 (top edge)
float width = rect.Width;   // 64
float height = rect.Height; // 32

float left = rect.Left;     // 100
float right = rect.Right;   // 164
float top = rect.Top;       // 200
float bottom = rect.Bottom; // 232

Vect2 center = rect.Center; // (132, 216)
Vect2 size = rect.Size;     // (64, 32)
```

Operations:

Operation	What It Does	Example
Contains(point)	Is a point inside?	rect.Contains(mousePos)
Contains(other)	Is another rect inside?	rect.Contains(otherRect)
Intersects(other)	Do they overlap?	rect.Intersects(wall)
Intersection(other)	Get the overlapping area	rect.Intersection(wall)
Union(other)	Smallest rect containing both	rect.Union(other)
Inflate(amount)	Grow or shrink	rect.Inflate(10)
Offset(amount)	Move the rectangle	rect.Offset(new Vect2(50, 0))
Move(position)	Set new position	rect.Move(new Vect2(100, 100))

Common Rectangle Patterns:

Collision detection:

```
// Simple overlap check
if (playerRect.Intersects(enemyRect))
{
    // Handle collision
}

// Check if a point is inside (useful for UI clicks)
if (buttonRect.Contains(mousePosition))
{
    // Clicked the button
}

// Resolve overlap (push objects apart)
if (playerRect.Intersects(wallRect))
{
    Vect2 push = CollisionHelper.PushRectRect(playerRect, wallRect);
    playerRect = playerRect.Offset(push);
}
```

Rendering with rectangles:

```
// Draw a sprite using a rectangle
batcher.Draw(texture, destRect, sourceRect, Color.White);

// Draw a UI panel
batcher.DrawRect(panelRect, panelColor);
```

Understanding Color

A `Color` represents red, green, blue, and alpha (transparency) values.

What you use it for:

- Tinting sprites
- Background colors
- UI colors
- Visual effects

Creating Colors:

```
// From RGB values
Color red = new Color(255, 0, 0);
Color custom = new Color(128, 64, 32);

// From float values (0-1)
Color half = new Color(0.5f, 0.5f, 0.5f);

// From hex string
Color fromHex = new Color("#FF8040"); // RGB
Color withAlpha = new Color("#80FF8040"); // ARGB

// Pre-defined colors
Color white = Color.White;
Color black = Color.Black;
Color red = Color.Red;
Color green = Color.Green;
Color blue = Color.Blue;
Color yellow = Color.Yellow;
Color cyan = Color.Cyan;
Color magenta = Color.Magenta;
Color orange = Color.Orange;
Color purple = Color.Purple;
Color pink = Color.Pink;
Color brown = Color.Brown;
Color gray = Color.Gray;
Color lightGray = Color.LightGray;
Color darkGray = Color.DarkGray;
Color gold = Color.Gold;
Color silver = Color.Silver;
Color navy = Color.Navy;
Color olive = Color.Olive;
Color teal = Color.Teal;
Color coral = Color.Coral;
Color crimson = Color.Crimson;
```

Operations:

Operation	What It Does	Example
Lerp(target, t)	Blend two colors	Color.Lerp(Color.Red, Color.Blue, 0.5f)
WithAlpha(alpha)	Set transparency	Color.Red.WithAlpha(0.5f)
* (multiply)	Scale or combine	Color.Red * 0.5f (darken)
+ (add)	Add colors	Color.Red + Color.Blue
- (subtract)	Subtract colors	Color.Red - Color.Blue
== / !=	Compare colors	if (color == Color.Red)

Common Color Patterns:

Tinting sprites:

```
// Tint a sprite red
batcher.Draw(texture, position, Color.Red);

// Half-transparent
batcher.Draw(texture, position, Color.White.WithAlpha(0.5f));
```

Color transitions:

```
// Fade from red to blue over time
float t = elapsed / duration;
Color currentColor = Color.Lerp(Color.Red, Color.Blue, t);
```

Darkening for night effects:

```
// Everything gets darker
Color dark = Color.White * 0.5f; // Half brightness
```

Understanding Easing Functions

Easing functions control how a value changes over time. They make animations feel more natural.

The Problem: If you just move something from A to B in a straight line, it feels robotic and stiff.

The Solution: Easing functions make things speed up, slow down, bounce, or spring.

Example:

```
// Without easing: linear movement
float progress = elapsed / duration;

// With easing: smooth acceleration and deceleration
float eased = Easing.Ease(EaseType.QuadInOut, progress);

// Use the eased value to interpolate
float currentValue = MathHelper.Lerp(startValue, endValue, eased);
```

The Easing Families:

Family	Description	Visual Feel
Linear	Constant speed	Mechanical, robotic
Quadratic	Moderate acceleration	Smooth, natural
Cubic	Strong acceleration	Punchy
Quartic	Very strong acceleration	Aggressive
Quintic	Extreme acceleration	Dramatic
Sine	Smooth, wavy	Gentle, organic
Exponential	Rapid acceleration	Explosive
Circular	Circular motion	Natural, curved
Back	Overshoots then corrects	Springy, playful
Elastic	Oscillates	Bouncy, rubbery
Bounce	Bounces at the end	Playful, energetic

What "In", "Out", "InOut", and "OutIn" Mean:

Suffix	What It Does	Visual
In	Starts slow, ends fast	 → 
Out	Starts fast, ends slow	 → 
InOut	Starts slow, speeds up, ends slow	 →  → 
OutIn	Starts fast, slows down, ends fast	 →  → 

When To Use Each:

Easing	Best For
QuadInOut	Default easing (most things)
QuadOut	UI elements (buttons sliding in)
QuadIn	UI elements (menus sliding out)
SineInOut	Camera movements, smooth transitions
ElasticOut	UI elements (bouncy notifications)
BounceOut	UI elements (jumping in)
BackOut	UI elements (overshoot effect)
ExpoIn	Explosions, dramatic entrances

MathHelper Utilities

MathHelper provides additional math utilities:

Function	What It Does
Lerp(a, b, t)	Linear interpolation
SmoothStep(a, b, t)	Smooth Hermite interpolation
InverseLerp(a, b, value)	Reverse of Lerp (gets t from value)
Remap(value, fromMin, fromMax, toMin, toMax)	Remap from one range to another
Clamp(value, min, max)	Restrict to a range
Saturate(value)	Clamp between 0 and 1
Wrap(value, min, max)	Wrap around a range
MoveTowards(current, target, maxDelta)	Move towards a target
DirectionToAngle(direction)	Convert vector to angle
AngleToDirection(angle)	Convert angle to vector
Snap(value, gridSize)	Snap to a grid
ToRadians(degrees)	Convert degrees to radians
ToDegrees(radians)	Convert radians to degrees

Examples:

```
// Remap health from 0-100 to 0-1
float healthPercent = MathHelper.Remap(health, 0, 100, 0, 1);

// Smoothly move value toward target
float newValue = MathHelper.MoveTowards(currentValue, targetValue, 5f);

// Convert between angles
float radians = MathHelper.ToRadians(90); // PI/2
float degrees = MathHelper.ToDegrees(PI); // 180

// Snap to a grid
float snapped = MathHelper.Snap(12.3f, 5f); // 10f
```

What's Next?

- [Master Rendering](#) — Draw sprites, text, shapes, and effects
- [Add Audio](#) — Play sounds with pooling and priority control
- [Handle Input](#) — Keyboard, mouse, gamepad, and action mapping

Next: [5. Rendering](#)

5. Rendering

Drawing sprites, text, shapes, and effects.

What Is Rendering?

Rendering is the process of putting images on the screen. Void handles all the low-level graphics work. You just tell it what to draw and where.

Without a rendering system, you'd have to talk directly to the graphics card. You'd have to manage textures, create vertex buffers, and handle the rendering pipeline. Void handles all of this so you can focus on your game.

Why You Need This

Drawing things on screen seems simple, but it gets complex fast. You need to:

- Load images from files
- Store them on the graphics card
- Draw them at the right position
- Handle transparency and blending
- Sort them so things appear in the right order
- Do all of this quickly (60 times per second)

Void handles all of this. You just call `Draw()`.

The Camera

A camera defines what part of your game world the player can see. It can move, zoom, and follow the player.

What the Camera Does:

- Moves around your game world (scrolling)
- Zooms in and out
- Converts between screen coordinates and world coordinates

Example:

```
// Create a camera
Camera camera = new Camera();

// Position the camera (follow the player)
camera.Position = player.Position;

// Zoom in
camera.Zoom = 2.0f; // 2x zoom

// Clamp the camera to the level bounds
camera.Bounds = new Rect2(0, 0, levelWidth, levelHeight);

// Convert mouse position to world coordinates
Vect2 worldPos = camera.ScreenToWorld(mousePosition);

// Convert world position to screen coordinates
Vect2 screenPos = camera.WorldToScreen(enemyPosition);
```

Camera in Rendering:

```
batcher.Begin(SortMode.BackToFront, BlendMode.Alpha, camera);
// All sprites are drawn relative to the camera
batcher.Draw(playerTexture, playerPosition, Color.White);
batcher.End();
```

Texture Atlasing (The AtlasManager)

This is one of Void's most important performance features.

The Problem: Every time you draw a sprite, the graphics card has to switch to that texture. Texture switches are expensive. If you draw 100 different sprites with 100 different textures, you get 100 draw calls. That's slow.

The Solution: Pack all your textures into one big texture called an atlas. Then all your sprites come from the same texture. The graphics card doesn't need to switch textures. You get 1 draw call instead of 100. This is dramatically faster.

How AtlasManager Works:

1. You load textures normally through `AssetManager`
2. `AtlasManager` automatically packs them into atlas pages
3. Each atlas page is a large texture (default 2048x2048)
4. The packer arranges sprites efficiently (Skyline or Guillotine)
5. When the atlas is full, it creates a new page
6. Old, unused textures are evicted (LRU) to free space
7. Defragmentation runs when fragmentation gets too high

You don't have to do anything. It just works.

```
// Load textures normally
Texture playerTexture = AssetManager.Instance.Load<Texture>("player.png");
Texture enemyTexture = AssetManager.Instance.Load<Texture>("enemy.png");
Texture bulletTexture = AssetManager.Instance.Load<Texture>("bullet.png");

// The AtlasManager packs them automatically when you draw
batcher.Draw(playerTexture, playerPos, Color.White);
batcher.Draw(enemyTexture, enemyPos, Color.White);
batcher.Draw(bulletTexture, bulletPos, Color.White);

// All three sprites are in the same atlas → one draw call
```

Manual Atlas Control:

```
// Check atlas usage
var metrics = AtlasManager.Instance.GetMetrics();
Console.WriteLine($"Atlas usage: {metrics.PercentageFull:F1}%");
Console.WriteLine($"Textures packed: {metrics.TextureCount}");
Console.WriteLine($"Pages used: {metrics.UsedPages}/{metrics.TotalPages}");

// Process defragmentation moves (call once per frame)
AtlasManager.Instance.ProcessPendingDefragMoves(maxMovesPerFrame: 5);
```

Atlas Packing Algorithms:

Packer	How It Works	Best For
SkylinePacker	Maintains a skyline of occupied pixels	Most games (efficient)
GuillotinePacker	Splits free space into rectangles	Simple implementation

To change the packer:

```
settings.SetAtlasPacker(typeof(SkylinePacker));  
// or  
settings.SetAtlasPacker(typeof(GuillotinePacker));
```

The SpriteBatcher

The SpriteBatcher is how you draw images (sprites) efficiently.

Why Use a Batch?

Drawing one sprite at a time is slow. Drawing hundreds or thousands at once is fast. The batcher collects all your draw commands and sends them to the graphics card in one go.

Example:

```
// Create a batcher
var batcher = new SpriteBatcher();

// Begin a batch
batcher.Begin(SortMode.BackToFront, BlendMode.Alpha, camera);

// Draw sprites
batcher.Draw(playerTexture, playerPosition, Color.White);
batcher.Draw(enemyTexture, enemyPosition, Color.Red);

// Draw a sprite with a source rectangle (spritesheet)
batcher.Draw(
    texture,
    new Rect2(200, 200, 64, 64),          // Destination
    new Rect2(0, 0, 32, 32),              // Source rectangle
    Color.White
);

// Draw a sprite with rotation and scale
batcher.Draw(
    texture,
    new Rect2(200, 200, 64, 64),
    Color.White,
    rotation: 0.5f,                       // Rotate by 0.5 radians
    scale: new Vect2(2, 2),                // Double the size
    origin: new Vect2(32, 32)              // Rotate around the center
);

// Draw text
batcher.DrawText(
    font,
    "Hello World!",
    new Vect2(300, 300),
    Color.Black,
    TextAlignment.Center
);

// End the batch (sends everything to the GPU)
batcher.End();
```

Draw Methods Overview:

Method	What It Does
<code>Draw(texture, position, color)</code>	Draw a sprite at a position
<code>Draw(texture, rect, color)</code>	Draw a sprite in a rectangle
<code>Draw(texture, rect, sourceRect, color)</code>	Draw a sprite with a source rectangle
<code>Draw(texture, rect, sourceRect, color, rotation, scale, origin, effects, depth)</code>	Full transformations
<code>DrawText(font, text, position, color)</code>	Draw text at a position
<code>DrawText(font, text, bounds, color)</code>	Draw text within bounds
<code>DrawBypassAtlas(...)</code>	Draw without atlas packing

SortMode:

Mode	What It Does	When To Use
<code>BackToFront</code>	Draw far objects first, near objects last	Most games (painter's algorithm)
<code>FrontToBack</code>	Draw near objects first, far objects last	When you have lots of opaque objects
<code>Immediate</code>	Draw in the order you call <code>Draw()</code>	When you need exact control
<code>Deferred</code>	Collect commands but don't draw yet	When you want to draw later

TextAlignment:

Alignment	What It Means
TopLeft	Text starts at the top-left of the position
TopCenter	Text is centered horizontally at the top
TopRight	Text ends at the top-right of the position
CenterLeft	Text is vertically centered, aligned left
Center	Text is centered both ways
CenterRight	Text is vertically centered, aligned right
BottomLeft	Text starts at the bottom-left
BottomCenter	Text is centered horizontally at the bottom
BottomRight	Text ends at the bottom-right

TextureEffects:

Effect	What It Does
None	Normal rendering
Horizontal	Flip horizontally (mirror left-right)
Vertical	Flip vertically (mirror up-down)
Horizontal Vertical	Flip both ways (180° rotation)

Nine-Patch (Scalable UI):

A nine-patch divides a texture into 9 regions: corners (fixed size), edges (stretch), and center (stretch). This lets you create UI elements that scale to any size without distorting the corners.

```
batcher.DrawNinePatch(  
    uiTexture,  
    new Rect2(100, 100, 200, 100),    // Destination  
    new Rect2(0, 0, 64, 64),           // Source  
    new Rect2(8, 8, 8, 8),             // Corners (left, top, right, bottom)  
    Color.White  
);
```


The PrimitiveBatcher

The `PrimitiveBatcher` draws simple shapes: lines, rectangles, circles, polygons, and triangles.

Example:

```
var batcher = new PrimitiveBatcher();
batcher.Begin(SortMode.BackToFront);

// Draw a filled rectangle
batcher.DrawRect(new Vect2(100, 100), new Vect2(200, 150), Color.Red);

// Draw a rectangle outline
batcher.DrawRectOutline(new Vect2(100, 100), new Vect2(200, 150), Color.Blue);

// Draw a filled circle
batcher.DrawCircle(new Vect2(300, 300), 50, Color.Green);

// Draw a circle outline
batcher.DrawCircleOutline(new Vect2(300, 300), 50, Color.Green, segments: 32);

// Draw a line
batcher.DrawLine(new Vect2(0, 0), new Vect2(100, 100), Color.White);

// Draw a polygon
Vect2[] vertices = { new(0,0), new(50,0), new(25,50) };
batcher.DrawPolygon(vertices, Color.Yellow);

batcher.End();
```

Draw Methods:

Method	What It Draws
<code>DrawRect</code>	Filled rectangle
<code>DrawRectOutline</code>	Rectangle outline
<code>DrawCircle</code>	Filled circle
<code>DrawCircleOutline</code>	Circle outline
<code>DrawLine</code>	Line between two points
<code>DrawLineStrip</code>	Connected lines
<code>DrawPoint</code>	Single point
<code>DrawPoints</code>	Multiple points
<code>DrawPolygon</code>	Filled polygon
<code>DrawPolygonOutline</code>	Polygon outline
<code>DrawTriangle</code>	Filled triangle
<code>DrawTriangleOutline</code>	Triangle outline

Blend Modes

Blend modes control how a sprite blends with the pixels behind it.

Blend Mode	What It Does	When To Use
Alpha	Standard transparency	Most sprites
Add	Adds the sprite to what's behind it	Glow effects, lights, fire
Multiply	Multiplies the sprite with what's behind it	Darkening, shadows
None	Overwrites what's behind it	UI elements, opaque objects
Min	Takes the darker of the two pixels	Darkening effects
Max	Takes the brighter of the two pixels	Lightening effects
Premultiplied	Optimized transparency	When you need extra performance

Example:

```
// Additive blending for a glow effect
batcher.Begin(blendMode: BlendMode.Add);
batcher.Draw(glowTexture, glowPosition, Color.White);
batcher.End();

// Normal blending for sprites
batcher.Begin(blendMode: BlendMode.Alpha);
batcher.Draw(playerTexture, playerPosition, Color.White);
batcher.End();
```

Custom Blend Mode:

```
var custom = BlendMode.Create(
    colorSrc: BlendFactor.SrcAlpha,
    colorDst: BlendFactor.One,
    colorEq: BlendEquation.Add,
    alphaSrc: BlendFactor.One,
    alphaDst: BlendFactor.OneMinusSrcAlpha,
    alphaEq: BlendEquation.Add
);

batcher.Begin(blendMode: custom);
```

Shaders

Void supports custom shaders loaded from .shader files.

Shader File Format:

```
[vertex]
#version 330 core
// Vertex shader source

[fragment]
#version 330 core
// Fragment shader source
```

Usage:

```
// Load a shader
var shader = AssetManager.Instance.Load<Shader>("shaders/glow.shader");

// Set uniforms
shader.SetUniform("uTime", 1.5f);
shader.SetUniform("uColor", Color.Red);
shader.SetUniform("uTexture", texture);

// Use the shader
batcher.SetShader(shader);
batcher.Begin();
// ... draw commands ...
batcher.End();
```

Post-Processing

Apply visual effects to the entire screen after your game is rendered.

```
// Create a post-processor
var bloomShader = AssetManager.Instance.Load<Shader>("shaders/bloom.shader");
var postProcessor = new PostProcessor(bloomShader, new Vect2(1920, 1080));

// In your render loop:
// 1. Render your game to a render target
// 2. Apply post-processing
postProcessor.Apply(sceneTarget, camera);

// 3. Draw the result
var result = postProcessor.GetResultTexture();
batcher.DrawBypassAtlas(result, new Rect2(0, 0, 1920, 1080), Color.White);
```

Render Targets

Render targets let you render off-screen, which is useful for post-processing, effects, and UI rendering.

```
// Get a render target from the pool
var target = RenderTarget.Get(1920, 1080, sRGB: true);

// Use the render target
target.Clear(Color.Transparent);
// ... draw operations ...
target.Display();

// Return to pool
RenderTarget.Return(target);

// Resize an existing target
target = RenderTarget.Resize(target, 1280, 720);
```

What's Next?

- [Add Audio](#) — Play sounds with pooling and priority control
- [Handle Input](#) — Keyboard, mouse, gamepad, and action mapping
- [Manage Assets](#) — Load, cache, and manage your game assets

Next: [6. Audio](#)

6. Audio

Adding sound to your game.

What Is the Audio System?

The audio system lets you play sounds in your game. It handles loading sound files, playing them, and managing multiple sounds at once.

Without this system, you'd have to manage sound buffers, handle concurrent playback, and deal with voice allocation yourself. Void handles all of this.

Why You Need This

Sounds are more complex than they seem. You need to:

- Load sound files from disk
- Store them in memory
- Play multiple sounds at once
- Control volume, pan, and pitch
- Handle sounds finishing or looping
- Manage memory when sounds aren't used

Void handles all of this. You just call `Play()`.

Sound Assets

First, you load a sound asset:

```
// Load a sound file
Sound explosionSound = AssetManager.Instance.Load<Sound>("explosion.wav");
Sound musicSound = AssetManager.Instance.Load<Sound>("background.ogg");
```

Playing Sounds

There are two ways to play a sound: one-shot and instance.

One-shot: Play a sound and forget about it.

```
// Play a sound once
explosionSound.PlayOneShot(volume: 0.8f, pan: 0f, pitch: 1f);

// Play with random pitch variation (sounds more natural)
explosionSound.PlayWithPitchVariation(pitchRange: 0.15f, volume: 0.8f);

// Play and automatically dispose when finished
explosionSound.PlayAndForget(volume: 0.8f);
```

Instance: Keep control of the sound while it plays.

```
// Create a sound instance
SoundInstance instance = sound.CreateInstance(SoundCategory.SFX);

// Configure it
instance.Volume = 0.8f;    // 0-1
instance.Pan = -0.5f;      // -1 (left) to 1 (right)
instance.Pitch = 1.2f;     // 0.1-10 (speed up or slow down)
instance.Looping = true;   // Loop forever

// Play it
instance.Play();

// Later, stop it
instance.Stop();

// Or pause and resume
instance.Pause();
instance.Play();

// Check its status
if (instance.IsPlaying) { /* ... */ }
if (instance.IsPaused) { /* ... */ }
if (instance.IsStopped) { /* ... */ }

// Get playback info
float playTime = instance.PlayTime;
float duration = instance.Duration;
float progress = instance.Progress; // 0-1

// Events
instance.SoundCompleted += (s, e) =>
{
    Console.WriteLine("Sound finished playing!");
};

instance.SoundLooped += (s, e) =>
{
    Console.WriteLine($"Sound looped {e.LoopCount} times");
};
```

Sound Categories

Categories group sounds so you can control their volume together.

Example:

```
// Define your categories
public enum SoundCategory
{
    Music,
    SFX,
    UI,
    Dialogue
}

// Set the volume for each category
SoundHelper.SetCategoryVolume(SoundCategory.Music, 0.5f);
SoundHelper.SetCategoryVolume(SoundCategory.SFX, 0.8f);
SoundHelper.SetCategoryVolume(SoundCategory.UI, 1.0f);
SoundHelper.SetCategoryVolume(SoundCategory.Dialogue, 0.9f);

// Play a sound with a category
sound.PlayOneShot(volume: 1f, pan: 0f, pitch: 1f, category: SoundCategory.SFX);
```

Master Volume

Control the overall volume of all sounds:

```
// Set master volume (0-1)
SoundHelper.MasterVolume = 0.7f;
```

SoundPriority

When many sounds play at once, some need to be more important than others.

Priority	When To Use
Low	Ambient sounds, background noise
Normal	Most gameplay sounds (footsteps, gunshots)
High	Important feedback (UI sounds)
Critical	Dialogue, quest updates, alarms

How It Works:

If the sound pool is full and a new sound wants to play:

1. It looks for any stopped instances to reuse
2. If the new sound has higher priority, it steals a lower-priority sound
3. If all else fails, it steals the oldest sound

Example:

```
// Load a sound with priority
Sound criticalSound = AssetManager.Instance.LoadSound("dialogue.wav",
SoundPriority.Critical);
Sound ambientSound = AssetManager.Instance.LoadSound("birds.wav",
SoundPriority.Low);
```

Sound Groups

Play random sounds from a group (great for footsteps, impact sounds, etc.):

```
// Register a sound group
SoundHelper.RegisterSoundGroup("footsteps", footstep1, footstep2, footstep3);

// Play a random footstep
SoundHelper.PlayFromGroup("footsteps", volume: 0.7f, pan: 0f, pitch: 1f);

// Play with random pitch variation
SoundHelper.PlayFromGroup(
    "footsteps",
    volume: 0.7f,
    pan: 0f,
    pitch: 1f,
    category: SoundCategory.SFX,
    withVariation: true,
    pitchRange: 0.15f
);
```

Batch Operations

Control all sounds at once:

```
// Stop all sounds
SoundHelper.StopAll();

// Pause all sounds
SoundHelper.PauseAll();

// Resume all sounds
SoundHelper.ResumeAll();

// Stop all sounds with a specific name
SoundHelper.StopAll("explosion.wav");

// Apply volume to all active sounds
SoundHelper.ApplyVolumeToAll(0.8f);
```

Common Audio Patterns

Footsteps:

```
// Register footstep sounds
SoundHelper.RegisterSoundGroup("footsteps", footstep1, footstep2, footstep3,
footstep4);

// In Update()
if (isWalking)
{
    _stepTimer += time.DeltaTime;
    if (_stepTimer > _stepInterval)
    {
        SoundHelper.PlayFromGroup("footsteps", volume: 0.5f, category:
SoundCategory.SFX);
        _stepTimer = 0f;
    }
}
```

Background Music:

```
// Start music
var musicInstance = _backgroundMusic.CreateInstance();
musicInstance.Looping = true;
musicInstance.Volume = 0.5f;
musicInstance.Play();

// Save reference to stop later
_musicInstance = musicInstance;

// OnExit() or when changing music
_musicInstance.Stop();
_musicInstance.Dispose();
```

Dynamic Sound Effects:

```
// Play a sound with pitch based on speed
float pitch = MathHelper.Lerp(0.8f, 1.2f, playerSpeed / maxSpeed);
explosionSound.PlayOneShot(volume: 0.8f, pan: 0f, pitch: pitch);
```

What's Next?

- [Handle Input](#) — Keyboard, mouse, gamepad, and action mapping
- [Manage Assets](#) — Load, cache, and manage your game assets
- [Learn About the Asset Packer](#) — Protect your assets with encryption

Next: [7. Input](#)

7. Input

Keyboard, mouse, gamepad, and action mapping.

What Is the Input System?

The input system reads what the player is doing with their keyboard, mouse, and gamepad. It gives you simple ways to check if a key is pressed, a button is clicked, or a joystick is moved.

Without this system, you'd have to talk directly to the operating system's input APIs. You'd have to handle each device separately and manage state tracking yourself. Void handles all of this.

Why You Need This

Input is more complex than it seems. You need to:

- Read from multiple devices (keyboard, mouse, gamepad)
- Track key states (pressed, held, released)
- Handle multiple gamepads with different button layouts
- Map inputs to game actions

Void handles all of this. You just check if a key or button is pressed.

Keyboard

Check if keys are pressed:

```
// Get the current keyboard state
var state = Keyboard.GetState();

// Check individual keys
if (state.IsKeyDown(KeyboardKey.W))
    MoveForward();

if (state.IsKeyDown(KeyboardKey.Space))
    Jump();

if (state.IsKeyDown(KeyboardKey.Escape))
    ExitGame();

// Check modifier combinations
if (state.IsKeyDown(KeyboardKey.LControl) && state.IsKeyDown(KeyboardKey.S))
    SaveGame();

// Get all pressed keys
var pressedKeys = state.GetPressedKeys();
int count = state.GetPressedKeyCount();

// Check lock states
bool capsLock = state.CapsLock;
bool numLock = state.NumLock;
```

KeyboardKey Categories:

Category	Keys
Letters	A-Z
Numbers	0-9
Function Keys	F1-F15
Navigation	Left, Right, Up, Down, PageUp, PageDown, Home, End
Modifiers	LControl, RControl, LShift, RShift, LAlt, RAlt
Numpad	Numpad0-9, Add, Subtract, Multiply, Divide
Other	Space, Enter, Backspace, Tab, Escape, Insert, Delete, Pause

Mouse

Check mouse buttons and position:

```
// Get the current mouse state
var state = Mouse.GetState();

// Check buttons
if (state.IsButtonPressed(MouseButton.Left))
    HandleClick();

if (state.IsButtonPressed(MouseButton.Right))
    OpenContextMenu();

if (state.IsButtonPressed(MouseButton.Middle))
    PanCamera();

// Get position
Vect2 position = state.Position;
float x = state.X;
float y = state.Y;

// Get scroll wheel delta
int scroll = state.ScrollWheel; // Positive = scroll up, negative = scroll down

// Set mouse position
Mouse.SetPosition(100, 200);
```

MouseButton:

Button	Description
Left	Left mouse button
Right	Right mouse button
Middle	Middle mouse button (scroll wheel click)
XButton1	First extended button (back button on gaming mice)
XButton2	Second extended button (forward button on gaming mice)

Gamepad

Support for up to four gamepads with SDL database mapping:

```
// Update all gamepads (call once per frame)
Gamepad.UpdateAll();

// Get state for player 1
var state = Gamepad.GetState(PlayerIndex.One);

if (state.IsConnected)
{
    // Check face buttons
    if (state.IsButtonPressed(GamepadButton.A))
        Jump();

    if (state.IsButtonPressed(GamepadButton.Start))
        PauseGame();

    // Get thumbstick positions
    Vect2 leftStick = state.LeftStick;
    Vect2 rightStick = state.RightStick;

    // Get trigger values (0-1)
    float leftTrigger = state.LeftTrigger;
    float rightTrigger = state.RightTrigger;

    // Check stick directions
    if (state.IsButtonPressed(GamepadButton.DPadUp))
        MoveUp();

    // Get force for specific directions
    float horizontal = state.GetForce(GamepadButton.LeftStickRight) -
        state.GetForce(GamepadButton.LeftStickLeft);

    float vertical = state.GetForce(GamepadButton.LeftStickDown) -
        state.GetForce(GamepadButton.LeftStickUp);
}
```

GamepadButton Categories:

Category	Buttons
Face	A, B, X, Y
DPad	DPadUp, DPadDown, DPadLeft, DPadRight
Shoulder	LeftShoulder, RightShoulder
Triggers	LeftTrigger, RightTrigger
Stick Press	LeftStick, RightStick
Stick Directions	LeftStickUp, LeftStickDown, LeftStickLeft, LeftStickRight, RightStickUp, RightStickDown, RightStickLeft, RightStickRight
System	Start, Back, Guide
Advanced	Paddle1-4, Touchpad, Misc1

Controller Mapping:

The system uses the SDL gamepad database to map controllers automatically. It supports Xbox, PlayStation, and Nintendo Switch controllers.

Button	Xbox	PlayStation	Nintendo Switch
A	A	Cross (×)	B
B	B	Circle (○)	A
X	X	Square (□)	Y
Y	Y	Triangle (△)	X

Input Actions

This is the most powerful part of Void's input system. Input Actions let you name your inputs and bind them to multiple keys/buttons. This makes your code cleaner and lets players remap controls easily.

The Problem:

```
// Without actions, your code is full of hardcoded keys
if (Keyboard.GetState().IsKeyDown(KeyboardKey.Space) ||
    Keyboard.GetState().IsKeyDown(KeyboardKey.Up) ||
    Gamepad.GetState().IsButtonPressed(GamepadButton.A))
{
    Jump(); // This works but it's messy
}
```

The Solution:

```
// Define actions once (do this in OnEnter or at startup)
InputAction.AddAction("Jump")
    .AddKey(KeyboardKey.Space)
    .AddKey(KeyboardKey.Up)
    .AddGamepad(GamepadButton.A);

InputAction.AddAction("MoveLeft")
    .AddKey(KeyboardKey.Left)
    .AddKey(KeyboardKey.A)
    .AddGamepad(GamepadButton.DPadLeft)
    .AddGamepad(GamepadButton.LeftStickLeft);

InputAction.AddAction("MoveRight")
    .AddKey(KeyboardKey.Right)
    .AddKey(KeyboardKey.D)
    .AddGamepad(GamepadButton.DPadRight)
    .AddGamepad(GamepadButton.LeftStickRight);

InputAction.AddAction("Pause")
    .AddKey(KeyboardKey.Escape)
    .AddGamepad(GamepadButton.Start);

// Then use them anywhere
var state = InputAction.GetState();

if (state.IsPressed("Jump"))
    StartJump();

if (state.IsHeld("MoveLeft"))
    MoveLeft();

if (state.IsReleased("Jump"))
    EndJump();

if (state.IsPressed("Pause"))
    TogglePause();
```

Action States:

State	What It Means
Pressed	The action was just pressed this frame (transition from Up to active)
Held	The action is being held down (active for multiple consecutive frames)
Released	The action was just released this frame (transition from active to Up)
Up	The action is not active

Why This Is Better:

1. **Cleaner code:** Your game logic uses names like "Jump" instead of hardcoded keys
2. **Multiple bindings:** One action can be triggered by keyboard, gamepad, or mouse
3. **Remappable:** You can change bindings without changing game code
4. **Zero-GC:** The system reuses dictionaries, so it doesn't create garbage
5. **Consistent state:** Pressed/Held/Released/Up detection works the same across all input types

Using Enums for Actions:

```
public enum GameAction
{
    Jump,
    MoveLeft,
    MoveRight,
    Pause
}

// Define actions using enums
InputAction.AddAction(GameAction.Jump)
    .AddKey(KeyboardKey.Space)
    .AddGamepad(GamepadButton.A);

// Use them
var state = InputAction.GetState();
if (state.IsPressed(GameAction.Jump))
    StartJump();
```

Remapping Controls:

```
// Get an action
var jumpAction = InputAction.GetAction("Jump");

// Clear existing bindings
jumpAction.ClearBindings();

// Add new bindings (player has remapped to Q)
jumpAction.AddKey(KeyboardKey.Q);

// Add mouse binding
jumpAction.AddMouse(MouseButton.Left);
```

Checking Action State in Detail:

```
var state = InputAction.GetState();

// Quick checks
if (state.IsPressed("Jump")) { /* Just pressed */ }
if (state.IsHeld("Sprint")) { /* Being held down */ }
if (state.IsReleased("Jump")) { /* Just released */ }
if (state.IsUp("Pause")) { /* Not pressed */ }

// Detailed state
var jumpState = state.GetState("Jump");
switch (jumpState)
{
    case ActionState.Pressed:
        StartJump();
        break;
    case ActionState.Held:
        ContinueJump();
        break;
    case ActionState.Released:
        EndJump();
        break;
    case ActionState.Up:
        // Not doing anything
        break;
}
```

What's Next?

- [Manage Assets](#) — Load, cache, and manage your game assets
- [Learn About the Asset Packer](#) — Protect your assets with encryption
- [Save/Load](#) — Save and load game data

Next: [8. Asset Management](#)

8. Asset Management

Loading, caching, mounts, and the asset pipeline.

What Is Asset Management?

Asset management is how you load and use images, sounds, fonts, and other content in your game.

Without this system, you'd have to load files from disk manually. You'd have to track which assets are loaded, manage memory, and handle reloading. Void handles all of this.

Why You Need This

Assets are the lifeblood of your game. You need to:

- Load images, sounds, fonts, and levels
- Keep them in memory so they load fast
- Unload them when they're not needed to save memory
- Load them from different sources (disk, packs, network)

Void handles all of this. You just ask for an asset and it gives it to you.

Loading Assets

Loading assets is simple:

```
// Load a texture
Texture playerTexture = AssetManager.Instance.Load<Texture>("player.png");

// Load a sound
Sound explosionSound = AssetManager.Instance.Load<Sound>("explosion.wav");

// Load a font
SpriteFont font = AssetManager.Instance.Load<SpriteFont>("fonts/arial.png");

// Load a shader
Shader bloomShader = AssetManager.Instance.Load<Shader>("shaders/bloom.shader");

// Load a spritesheet
Spritesheet spritesheet =
AssetManager.Instance.Load<Spritesheet>("sprites/player.sheet");

// Load an LDtk level
LDtkMap level = AssetManager.Instance.Load<LDtkMap>("levels/level.ldtk");
```

Supported Asset Types:

Asset Type	File Extensions	What It Is
Texture	.png, .bmp, .tga, .jpg, .gif, .psd, .hdr, .pic, .pnm	Images
Sound	.wav, .ogg, .mp3, .flac, .aiff, .au, .raw, .paf, .svx, .nist, .voc, .ircam, .w64, .mat4, .mat5, .pvf, .htk, .sds, .avr, .sd2, .caf, .wve, .mpc2k, .rf64	Audio
Shader	.shader	Graphics shaders
SpriteFont	.png, .bmp, .tga, .jpg, .gif, .psd, .hdr, .pic, .pnm	Bitmap fonts
Spritesheet	.sheet, .json	Sprite definitions
LDtkMap	.ldtk, .json	LDtk level data

How Assets Are Found

Void searches for assets in your Content folder. It looks through all subfolders.

```
Content/  
├── Textures/player.png    // Load as "Textures/player.png"  
├── Sounds/explosion.wav   // Load as "Sounds/explosion.wav"  
└── Fonts/arial.png       // Load as "Fonts/arial.png"
```

Asset Caching

Assets are automatically cached. Loading the same asset twice gives you the same instance.

```
Texture tex1 = AssetManager.Instance.Load<Texture>("player.png");  
Texture tex2 = AssetManager.Instance.Load<Texture>("player.png");  
  
// tex1 and tex2 are the same object
```

Asset Eviction

Assets are automatically unloaded after a period of inactivity. This keeps memory usage low.

```
// Configure eviction in settings
settings.SetAssetEviction(30)           // Unload after 30 minutes idle
        .SetAssetCheckIntervalMinutes(1); // Check every 1 minute
```

How Eviction Works:

1. Every minute (configurable), the AssetManager checks all loaded assets
2. If an asset hasn't been accessed for longer than the eviction timeout, it's unloaded
3. The asset data stays in the cache (raw bytes remain)
4. If the asset is accessed again, it's reloaded automatically
5. This keeps memory usage low while making reloading fast

The Mount System

Mounts are virtual file systems. They let you load assets from different sources.

Built-in Mounts:

Mount	What It Does
VirtualFilesystemMount	Loads assets from your Content folder
MacOSMount	Loads assets from macOS app bundles
PackMount	Loads assets from encrypted pack files
MacOSPackMount	Loads packs from macOS app bundles

How Mounts Work:

1. You add mounts to the AssetManager
2. When you load an asset, the AssetManager searches mounts in order
3. The first mount that has the file provides the data
4. This lets you layer assets (overwrite files, add mods, etc.)

Mount Priority:

Mounts are searched in the order they were added (or inserted). The first mount that contains the requested file provides the asset data.

```
// Add a mount with high priority (searched first)
AssetManager.Instance.AddMountToStart(new MyCustomMount());

// Add a mount with low priority (searched last)
AssetManager.Instance.AddMountToEnd(new MyCustomMount());

// Insert a mount at a specific position
AssetManager.Instance.InsertMount(2, new MyCustomMount());
```

Adding a Custom Mount:

```
public class NetworkMount : IMount
{
    public string Name => "Network Assets";

    public bool HasFile(string virtualPath)
    {
        // Check if the file exists on the network
        return NetworkAssetExists(virtualPath);
    }

    public byte[] ReadFile(string virtualPath)
    {
        // Download the file from the network
        return NetworkDownload(virtualPath);
    }
}

AssetManager.Instance.AddMountToStart(new NetworkMount());
```

Custom Asset Types

You can add your own asset types by implementing `IAAsset`.

Step 1: Implement `IAAsset`

```
public class MyCustomAsset : IAAsset
{
    public uint Id { get; }
    public string Tag { get; }
    public byte[] Data { get; }
    public bool IsValid { get; private set; }
    public AssetType Type => AssetType.Normal;
    public DateTime LastAccessTime { get; set; }

    public MyCustomAsset(uint id, byte[] data, string tag)
    {
        Id = id;
        Data = data;
        Tag = tag;
    }

    public void Load()
    {
        // Parse the data into your asset
        // For example: load a custom file format
        IsValid = true;
    }

    public void Unload()
    {
        // Free resources
        IsValid = false;
    }

    public void Dispose()
    {
        // Clean up
        IsValid = false;
    }
}
```

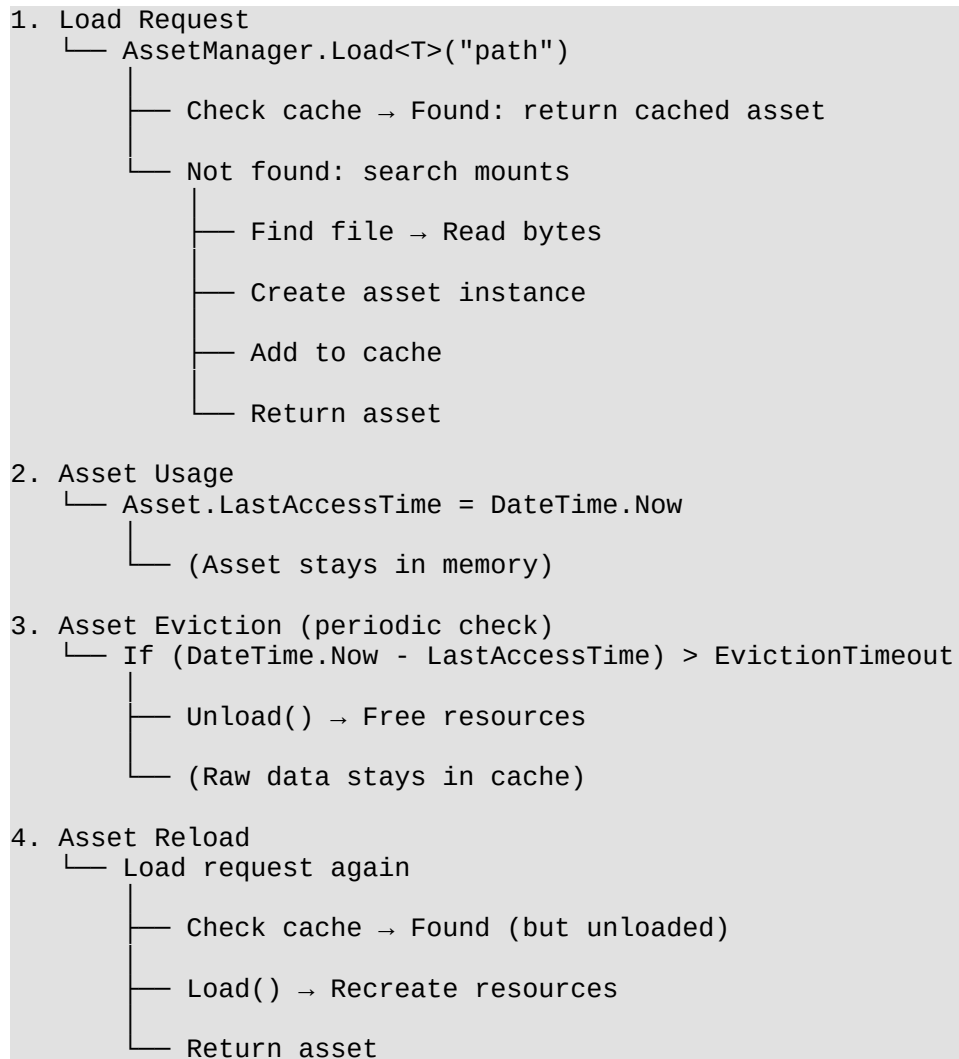
Step 2: Register the Asset Type

```
// In your game initialization
AssetManager.Instance.RegisterAssetType<MyCustomAsset>(
    extensions: new[] { ".myasset", ".custom" },
    factory: (id, data, tag) => new MyCustomAsset(id, data, tag)
);
```

Step 3: Load Your Custom Asset

```
MyCustomAsset myAsset =
AssetManager.Instance.Load<MyCustomAsset>("myfile.myasset");
```

The Asset Lifecycle



What's Next?

- [Learn About the Asset Packer](#) — Protect your assets with encryption
- [Asset Packer CLI](#) — Command-line tool for building packs
- [Save/Load](#) — Save and load game data

Next: [9. Asset Packer](#)

9. Asset Packer

Encrypted, compressed asset archives.

What Is the Asset Packer?

The Asset Packer takes all your game assets and packs them into a single encrypted file. This protects your work from theft and makes distribution simpler.

Why You Need This

When you release a game, your assets are your intellectual property. Without protection, anyone can extract your art, music, and levels.

The problem:

1. Someone buys your game
2. They use extraction tools to open your asset files
3. They pull out every image, sound file, and level
4. They list your assets on asset stores as their own
5. Your work is stolen and sold without your permission

This happens constantly. Most engines leave your assets exposed.

The solution:

Void packs your assets into encrypted archives. Without the key, the pack is just random bytes. Extraction tools don't work.

When to Pack

- **During development:** Work with loose assets in your Content folder. It's faster and easier.
- **For release:** Pack everything into encrypted archives. Protects your work.

How Packing Works

```
Content/          →  GameAssets.pack
├── Textures/
│   ├── player.png
│   └── enemy.png
├── Sounds/
│   └── explosion.wav
└── Levels/
    └── level.ldtk

Key file: GameAssets.key    // Without this, the pack is unreadable
```

The diagram illustrates the process of packing game assets. On the left, a directory structure under 'Content/' shows folders for 'Textures/' (containing 'player.png' and 'enemy.png'), 'Sounds/' (containing 'explosion.wav'), and 'Levels/' (containing 'level.ldtk'). An arrow points from this directory to a file named 'GameAssets.pack'. To the right of the arrow, the contents of the pack are listed: 'Encrypted header', 'Encrypted data', and 'CRC32 checksums'. At the bottom, a note states 'Key file: GameAssets.key' followed by a comment '// Without this, the pack is unreadable'.

Security Features

Feature	What It Does
AES-GCM 256-bit	Government-grade encryption. The same standard used by militaries.
Separate header encryption	File names and sizes are hidden. Attackers can't see what's in the pack.
CRC32 checksums	Detects tampering. If a file is modified, the checksum fails.
Per-chunk authentication	Each chunk has its own integrity check. Tampering with part of the pack is detected.
Adaptive compression	Only compresses when beneficial. PNG and OGG are stored as-is. JSON and WAV are compressed.

Chunked Encryption

For large packs, Void uses chunked encryption. The data section is split into chunks (default 1MB). Each chunk is encrypted separately.

Benefits:

- Reading a single file only decrypts the chunk containing that file
- You don't need to decrypt the entire pack to access one file
- Memory usage = header size + current chunk size (not pack size)
- Handles 1000+ packs without resource exhaustion

Loading a Pack

```
// Load the pack
var pack = AssetManager.Instance.LoadPack("GameAssets.pack");

// Add it to the search order
AssetManager.Instance.AddMountToStart(pack);

// Now all assets in the pack are available
Texture playerTexture = AssetManager.Instance.Load<Texture>("Textures/player.png");
```

The key file (`GameAssets.key`) is auto-detected if it's next to the pack file.

Multiple Packs

You can split your assets across multiple packs:

```
// Load multiple packs
var graphicsPack = AssetManager.Instance.LoadPack("Graphics.pack");
var audioPack = AssetManager.Instance.LoadPack("Audio.pack");
var levelsPack = AssetManager.Instance.LoadPack("Levels.pack");

// Add them to the search order
AssetManager.Instance.AddMountToStart(graphicsPack);
AssetManager.Instance.AddMountToStart(audioPack);
AssetManager.Instance.AddMountToStart(levelsPack);
```

Indexed Packs:

For large projects, packs can be split into multiple files:

```
GameAssets-0.pack
GameAssets-1.pack
GameAssets-2.pack
```

LoadAllPacks discovers and loads all indexed packs in a directory:

```
var packs = AssetManager.Instance.LoadAllPacks("Packs/");
foreach (var pack in packs)
{
    AssetManager.Instance.AddMountToEnd(pack);
}
```

Error Handling

Pack loading never throws for runtime use:

```
if (!Packer.TryLoadPack("Mod.pack", out var reader, out var error))
{
    Console.WriteLine($"Failed to load mod: {error}");
    return;
}
```

Error Codes:

Error	What It Means
PackNotFound	The pack file wasn't found
InvalidKey	The encryption key is wrong
UnsupportedVersion	The pack was created with a newer version
HeaderCorrupted	The pack header is corrupted
ChunkCorrupted	A chunk failed decryption (tampered or wrong key)
FileNotFound	The file wasn't found in the pack
ChecksumMismatch	The file's CRC32 checksum doesn't match

Key Management

The key is stored separately from the pack. You decide how to distribute it:

- **Embed in the game executable** — Most secure, but cannot be changed without rebuilding
- **Download from a CDN** — Can be updated or revoked, but requires internet
- **Store on a secure server** — Most flexible, but requires authentication
- **Distribute with the game** — Simple, but can be discovered

Security Note: No system is 100 percent secure. AES-GCM 256-bit encryption makes extraction extremely difficult, but a determined attacker with enough time and resources could theoretically find the key. The goal is to stop casual theft and make it harder to steal your work than it is to steal from other engines.

What's Next?

- [Asset Packer CLI](#) — Command-line tool for building packs
- [Save/Load](#) — Save and load game data
- [Pathfinding](#) — AI movement and navigation

Next: [10. Asset Packer CLI](#)

10. Asset Packer CLI

Command-line tool for building packs.

What Is the CLI Tool?

The CLI tool is a command-line program that turns your Content folder into encrypted pack files. You run it when your game is finished.

Why You Need This

You need a way to turn your loose assets into encrypted packs. The CLI tool does this. It's separate from the engine so you can run it as part of your build process.

Building a Pack

The most common command:

```
packcli build --content ./Content --output ./Output --name GameAssets
```

This creates `GameAssets.pack` and `GameAssets.key` in the Output folder.

CLI Commands

Command	What It Does
build	Build packs from your Content folder
extract	Extract all files from a pack
verify	Check if a pack is valid
list	List all files in a pack
update	Update an existing pack

Build Command Options

```
packcli build [options]
```

Option	What It Does	Default
--content	Your Content folder	Required
--output	Where to save packs	Required
--name	Base name for pack files	GameAssets
--include	Files to include (glob patterns)	All files
--exclude	Files to exclude (glob patterns)	None
--encrypt	Enable encryption	true
--compress	Compression algorithm	Deflate
--adaptive	Skip compression for some files	true
--max-files	Max files per pack	65535
--compression-level	Compression quality (1-9)	6
--chunk-size	Chunk size in KB	1024

Examples:

```
# Build with specific file patterns
packcli build --content ./Content --output ./Output --include "**/*.png" --include "**/*.wav"

# Build without encryption (faster, less secure)
packcli build --content ./Content --output ./Output --encrypt false

# Build with Brotli compression (smaller files, slower)
packcli build --content ./Content --output ./Output --compress Brotli

# Build with very high compression (smallest files)
packcli build --content ./Content --output ./Output --compression-level 9

# Exclude backup files
packcli build --content ./Content --output ./Output --exclude "**/Backup/**"
```

Extract Command

```
packcli extract --pack GameAssets.pack --output ./Extracted
```

Option	What It Does	Default
--pack	Pack file to extract	Required
--output	Where to extract files	Required
--key	Key file (if not auto-detected)	Auto-detect

Verify Command

```
packcli verify --pack GameAssets.pack
```

Checks if the pack is valid and hasn't been tampered with.

Option	What It Does	Default
--pack	Pack file to verify	Required
--key	Key file (if not auto-detected)	Auto-detect

List Command

```
packcli list --pack GameAssets.pack --detailed
```

Shows all files in the pack.

Option	What It Does	Default
--pack	Pack file to list	Required
--key	Key file (if not auto-detected)	Auto-detect
--detailed	Show file sizes and CRC	false

Update Command

```
packcli update --pack GameAssets.pack --add ./NewAssets --remove oldfile.txt
```

Adds or removes files from an existing pack.

Option	What It Does	Default
--pack	Pack to update	Required
--add	Files/folders to add	None
--remove	Files to remove	None
--key	Key file (if not auto-detected)	Auto-detect
--output	Output path for updated pack	Overwrite

Glob Patterns

The CLI supports glob patterns for include and exclude:

Pattern	What It Matches
*	Any characters except directory separators
**	Any characters including directory separators
*.png	All PNG files in any directory
/Backup/	Anything inside a Backup folder
Textures/*.png	PNG files directly in the Textures folder
**/Secret/*.wav	WAV files in any Secret folder

CI/CD Integration

The CLI tool is designed to work in build pipelines:

```
packcli build --content ./Content --output ./Build --name GameAssets
```

What's Next?

- [Save/Load](#) — Save and load game data
- [Pathfinding](#) — AI movement and navigation
- [Coroutines](#) — Code that runs over time

Next: [11. Save/Load System](#)

11. Save/Load System

Encrypted save data with manifest verification.

What Is the Save/Load System?

The save/load system lets you save game progress and load it later. It encrypts the save data so players can't cheat by editing save files.

Why You Need This

Every game needs to save progress. But saving is more complex than it seems:

- You need to write data to disk
- You need to read it back later
- You need to handle different versions of save files
- You need to prevent players from cheating by editing save files
- You need to detect corrupted save files

Void handles all of this. You just define what data to save and how to read it back.

How It Works

1. You define the data you want to save
2. You create a class that inherits from `ContentTypeWriterReader<T>`
3. You implement `Write` and `Read` methods
4. You call `Save()` and `Load()` methods

Basic Example

Define your save data:

```
public class PlayerSaveData
{
    public string Name { get; set; }
    public int Level { get; set; }
    public Vect2 Position { get; set; }
    public int Health { get; set; }
    public List<Item> Inventory { get; set; }
}
```

Create the save system:

```
public class PlayerSaveSystem : ContentTypeWriterReader<PlayerSaveData>
{
    // Constructor with encryption key
    public PlayerSaveSystem(string key) : base(key) { }

    // Write your data
    protected override void Write(PlayerSaveData data, ContentWriter writer)
    {
        writer.Write(data.Name);
        writer.Write(data.Level);
        writer.Write(data.Position);
        writer.Write(data.Health);
        writer.WriteObject(data.Inventory); // Complex objects are serialized
    }

    // Read your data
    protected override PlayerSaveData Read(ContentReader reader)
    {
        return new PlayerSaveData
        {
            Name = reader.ReadString(),
            Level = reader.ReadInt32(),
            Position = reader.ReadVect2(),
            Health = reader.ReadInt32(),
            Inventory = reader.ReadObject<List<Item>>()
        };
    }
}
```

Use the save system:

```
// Create the save system
var saveSystem = new PlayerSaveSystem("mySecretKey");

// Save
var data = new PlayerSaveData
{
    Name = "Player1",
    Level = 5,
    Position = new Vect2(100, 200),
    Health = 100,
    Inventory = new List<Item> { new Item("Sword"), new Item("Shield") }
};

saveSystem.Save("player.sav", data);

// Load with error handling
if (saveSystem.TryLoad("player.sav", out var loadedData, out var error))
{
    Console.WriteLine($"Loaded: {loadedData.Name}, Level: {loadedData.Level}");
}
else
{
    Console.WriteLine($"Load failed: {error}");
}
```

What You Can Save

Type	How To Write	How To Read
string	<code>writer.Write(string)</code>	<code>reader.ReadString()</code>
int	<code>writer.Write(int)</code>	<code>reader.ReadInt32()</code>
float	<code>writer.Write(float)</code>	<code>reader.ReadSingle()</code>
bool	<code>writer.Write(bool)</code>	<code>reader.ReadBoolean()</code>
byte	<code>writer.Write(byte)</code>	<code>reader.ReadByte()</code>
long	<code>writer.Write(long)</code>	<code>reader.ReadInt64()</code>
double	<code>writer.Write(double)</code>	<code>reader.ReadDouble()</code>
Vect2	<code>writer.Write(Vect2)</code>	<code>reader.ReadVect2()</code>
Rect2	<code>writer.Write(Rect2)</code>	<code>reader.ReadRect2()</code>
Color	<code>writer.Write(Color)</code>	<code>reader.ReadColor()</code>
object	<code>writer.WriteObject(object)</code>	<code>reader.ReadObject<T>()</code>

Security Features

Feature	What It Does
AES-GCM encryption	Encrypts the save data so players can't edit it
PBKDF2 key derivation	Strengthens your password/key against brute force
Manifest verification	Ensures the read order matches the write order
Version checking	Prevents loading old save files from different game versions
Atomic writes	Uses temp files to prevent corruption
CRC checks	Detects if the file has been modified

Versioning

The save system includes version checking:

```
// The version is stored in the save file
// When you change your save data format, increment the version
settings.SetAppVersion(1, 1, 0, 0); // New version

// The save system will reject older saves
// This prevents loading incompatible data
```

SaveError Codes

Error	What It Means
FileNotFound	The save file doesn't exist
WrongKey	The encryption key is wrong
VersionMismatch	The save file is from a different version
CorruptData	The save file is damaged
ManifestMismatch	The read order doesn't match the write order
InvalidPath	The file path is invalid
OutOfSpace	Not enough disk space

Best Practices

1. Use Try methods for loading:

```
if (saveSystem.TryLoad("player.sav", out var data, out var error))
{
    // Use the data
}
else
{
    // Handle the error
    if (error == SaveError.FileNotFound)
    {
        // Create a new save
    }
    else if (error == SaveError.WrongKey)
    {
        // Key has changed, player might be cheating
    }
}
```

2. Save frequently:

```
// Save after important events
protected override void OnUpdate(FrameTime time)
{
    if (playerHealthChanged || playerLevelChanged)
    {
        saveSystem.Save("player.sav", playerData);
    }
}

// Save on exit
protected override void OnExit()
{
    saveSystem.Save("player.sav", playerData);
}
```

3. Use versioning for evolving save data:

```
public class PlayerSaveData
{
    public int Version { get; set; } = 1;
    // ... other fields

    // In your Read method, check the version
    protected override PlayerSaveData Read(ContentReader reader)
    {
        int version = reader.ReadInt32();

        if (version == 1)
        {
            // Read version 1 format
        }
        else if (version == 2)
        {
            // Read version 2 format
        }
    }
}
```

What's Next?

- [Pathfinding](#) — AI movement and navigation
- [Coroutines](#) — Code that runs over time
- [State Machine](#) — Manage game states

Next: [12. Pathfinding](#)

12. Pathfinding

A, Dijkstra, BFS, Greedy, and flow fields.*

What Is Pathfinding?

Pathfinding is how you find a path from point A to point B in a game world. It's used for enemies chasing the player, NPCs moving around, and navigation in general.

Why You Need This

If you want enemies to chase the player, you need them to find a path through obstacles. Without pathfinding, enemies would walk into walls or get stuck. You could write your own pathfinding algorithm, but it's complex and easy to get wrong.

Void provides pathfinding so you don't have to write it yourself.

How It Works

1. You define points (nodes) in your world
2. You connect them (edges)
3. You ask the system to find a path from one point to another
4. The system returns a list of points to follow

Basic Example

```
var astar = new AStar2D();

// Add points
astar.AddPoint(0, new Vect2(0, 0));
astar.AddPoint(1, new Vect2(50, 0));
astar.AddPoint(2, new Vect2(100, 0));
astar.AddPoint(3, new Vect2(100, 50));
astar.AddPoint(4, new Vect2(50, 50));

// Connect points
astar.ConnectPoints(0, 1);
astar.ConnectPoints(1, 2);
astar.ConnectPoints(2, 3);
astar.ConnectPoints(3, 4);
astar.ConnectPoints(4, 0);

// Find a path from point 0 to point 3
var path = astar.GetPath(0, 3);

foreach (var point in path)
{
    Console.WriteLine($"Move to {point}");
}
```

Pathfinding Algorithms

Algorithm	What It Does	When To Use
AStar	Finds the shortest path using a heuristic	Most games (best balance)
Dijkstra	Finds the shortest path without a heuristic	When all paths cost the same
BFS	Finds the fastest path, ignoring costs	When all edges are equal
GreedyBestFirst	Finds a path quickly, may not be optimal	When you need speed over accuracy

AStar is the default for most games. It uses a heuristic (best guess) to guide the search, making it fast and accurate.

Dijkstra is AStar without a heuristic. It finds the shortest path but is slower.

BFS is the fastest but ignores costs. Good for simple maps where all moves cost the same.

GreedyBestFirst is the fastest but may not find the optimal path. Good when you need a path quickly.

Diagonal Movement

Mode	What It Does
Always	Always allow diagonal movement
Never	Never allow diagonal movement
AtLeastOneWalkable	Allow diagonal if at least one adjacent cell is walkable
OnlyIfNoObstacles	Allow diagonal only if both adjacent cells are walkable

Example:

```
// Use default diagonal mode
astar.DefaultDiagonalMode = DiagonalMode.OnlyIfNoObstacles;

// Or specify per path
var path = astar.GetPath(startId, endId, diagonalMode: DiagonalMode.Always);
```

Heuristics

Heuristic	What It Does	Best For
Manhattan	$ dx + dy $	4-directional movement (no diagonals)
Euclidean	$\sqrt{dx^2 + dy^2}$	8-directional movement (true distance)
Octile	$\max(dx , dy) + (\sqrt{2}-1) * \min(dx , dy)$	8-directional with diagonal cost
Chebyshev	$\max(dx , dy)$	8-directional with equal costs

Example:

```
astar.DefaultHeuristic = Heuristic.Octile;  
  
// Or specify per path  
var path = astar.GetPath(startId, endId, heuristic: Heuristic.Euclidean);
```

Weighted Paths

You can assign weights to points to influence pathfinding:

```
// Add a point with a weight  
astar.AddPoint(0, new Vect2(0, 0), weightScale: 1.0f); // Normal cost  
astar.AddPoint(1, new Vect2(50, 0), weightScale: 2.0f); // Double cost (avoid this area)  
astar.AddPoint(2, new Vect2(100, 0), weightScale: 0.5f); // Half cost (prefer this area)  
  
// Update weight later  
astar.SetPointWeightScale(1, 3.0f); // Make it even more costly
```

Disabling Points (Dynamic Obstacles)

You can disable points at runtime for dynamic obstacles:

```
// Disable a point (obstacle appears)  
astar.SetPointDisabled(obstacleNodeId, disabled: true);  
  
// Re-enable a point (obstacle removed)  
astar.SetPointDisabled(obstacleNodeId, disabled: false);
```

Extending AStar

Void allows you to override key pathfinding behaviors:

```
// Override cost calculation
astar.ComputeCostOverride = (fromId, toId) =>
{
    // Custom cost based on terrain type
    float baseCost = Vect2.Distance(astar.GetPointPosition(fromId),
astar.GetPointPosition(toId));
    return baseCost * terrainCosts[toId];
};

// Override heuristic estimation
astar.EstimateCostOverride = (fromId, toId) =>
{
    // Custom heuristic
    return Vector2.Distance(astar.GetPointPosition(fromId),
astar.GetPointPosition(toId)) * 0.8f;
};

// Filter neighbors (skip certain connections)
astar.FilterNeighborOverride = (currentId, neighborId) =>
{
    // Skip this neighbor if it's dangerous
    return IsDangerous(neighborId);
};
```

Flow Fields

Flow fields are for many agents moving toward the same target. Instead of each agent calculating its own path, you calculate the field once and all agents use it.

When to use flow fields:

- RTS games with many units moving to the same point
- Crowd simulations
- Tower defense games (many enemies following the same path)
- Any situation with many agents sharing a target

```
// Compute a flow field toward the target
var flowField = astar.ComputeFlowField(targetId);

// All agents use the same field
foreach (var agent in agents)
{
    var direction = flowField.GetDirection(agent.CurrentNode);
    if (direction != Vect2.Zero)
        agent.Move(direction);
}
```

FlowField API:

Method	What It Does
GetNextNode(currentNodeId)	Get the next node to move toward
GetDirection(currentNodeId)	Get the direction to move
HasNode(currentNodeId)	Check if the node is reachable
Count	Number of reachable nodes

Common Pathfinding Patterns

Chasing the player:

```
// Each frame, find the closest point to the player
int enemyNode = astar.GetClosestPoint(enemyPosition);
int playerNode = astar.GetClosestPoint(playerPosition);

// Find a path
var path = astar.GetPath(enemyNode, playerNode);

// Move along the path
if (path.Count > 0)
{
    Vect2 nextPosition = path[0];
    enemy.MoveToward(nextPosition);
}
```

Patrol paths:

```
// Pre-define a patrol route
int[] patrolNodes = { 0, 1, 2, 3, 4 };
int currentPatrolIndex = 0;

// Move to the next patrol point
int targetNode = patrolNodes[currentPatrolIndex];
var path = astar.GetPath(currentNode, targetNode);

// When you reach the target, advance
if (currentNode == targetNode)
{
    currentPatrolIndex = (currentPatrolIndex + 1) % patrolNodes.Length;
}
```

Avoiding dynamic obstacles:

```
// When an obstacle appears, disable its nodes
foreach (var obstacleNode in obstacleNodes)
{
    astar.SetPointDisabled(obstacleNode, true);
}

// Recalculate path
var path = astar.GetPath(startNode, endNode);
```

Performance Tips

1. **Reserve space upfront:** If you know how many nodes you'll have, reserve space

```
var astar = new AStar2D(initialCapacity: 1000);
```

2. **Reuse path objects:** Paths are lists. If you're recalculating often, reuse the list

```
var path = new List<Vect2>();
path = astar.GetPointPath(startId, endId);
```

3. **Use flow fields for many agents:** One calculation for all agents is faster than calculating individually

4. **Use distance squared for comparisons:** It's faster than distance

```
// Faster
if (position.DistanceSquared(target) < 100 * 100)
// Slower
if (position.Distance(target) < 100)
```

What's Next?

- [Coroutines](#) — Code that runs over time
- [State Machine](#) — Manage game states
- [Beacons](#) — Decoupled communication

Next: [13. Coroutines](#)

13. Coroutines

Code that runs over time.

What Are Coroutines?

Coroutines let you write code that runs over multiple frames. Instead of doing everything in one frame, you can spread it out.

Why You Need This

Many things in games take time. You need to:

- Wait for something to happen
- Animate something over time
- Sequence actions one after another
- Wait for a condition to be true

Without coroutines, you'd have to manage timers and state machines manually. Coroutines make this easy.

Without Coroutines

```
float timer = 0f;
bool done = false;

void Update(float dt)
{
    if (done) return;

    timer += dt;
    if (timer >= 2f)
    {
        Console.WriteLine("Done!");
        done = true;
    }
}
```

With Coroutines

```
IEnumerator MyCoroutine()
{
    yield return new WaitForSeconds(2f);
    Console.WriteLine("Done!");
}

// Start it
CoroutineManager.Instance.Run(MyCoroutine());
```

Basic Example

```
IEnumerator MyCoroutine()
{
    // Wait 1 second
    yield return 1.0f;

    // Do something
    Console.WriteLine("After 1 second");

    // Wait 0.5 seconds
    yield return 0.5f;

    Console.WriteLine("Done!");
}

// Start a coroutine
CoroutineManager.Instance.Run(MyCoroutine());

// Start with initial delay
CoroutineManager.Instance.Run(0.5f, MyCoroutine());

// Stop a coroutine
var handle = CoroutineManager.Instance.Run(MyCoroutine());
CoroutineManager.Instance.Stop(handle);
```

Tweens (Animations)

Tweens are animations that interpolate a value over time.

```
// Animate a position
var tween = new Tween<float>(
    from: 0f,
    to: 100f,
    duration: 1f,
    type: EaseType.QuadOut,
    lerpFunc: (a, b, t) => MathHelper.Lerp(a, b, t),
    onUpdate: value => position.X = value
);

CoroutineManager.Instance.Run(tween);
```

Tween Types:

Tween	What It Does
Tween<T>	Basic animation from A to B
CallbackTween<T>	Runs a callback when done
DelayedTween<T>	Starts after a delay
LoopTween<T>	Repeats N times or forever
PingPongTween<T>	Goes back and forth
PulseTween<T>	Pulses like a heartbeat
SpeedTween<T>	Adjust speed (slow-mo/fast-forward)

Examples:

```
// Delayed tween (starts after 0.5 seconds)
var delayed = new DelayedTween<float>(
    0f, 100f, 1f, EaseType.QuadOut, Lerp,
    value => x = value,
    delay: 0.5f
);

// Loop tween (repeats 3 times)
var loop = new LoopTween<float>(
    0f, 100f, 1f, EaseType.QuadOut, Lerp,
    value => x = value,
    loops: 3
);

// Infinite loop tween
var infinite = new LoopTween<float>(
    0f, 100f, 1f, EaseType.QuadOut, Lerp,
    value => x = value,
    loops: -1
);

// Ping-pong (oscillates)
var pingpong = new PingPongTween<float>(
    0f, 100f, 1f, EaseType.QuadOut, Lerp,
    value => x = value
);

// Pulse (heartbeat effect)
var pulse = new PulseTween<float>(
    1f, 1.5f, 1f, EaseType.QuadOut, Lerp,
    value => scale = value,
    cycles: -1 // Infinite
);
```

Compositions

Run multiple coroutines together:

Sequence (one after another):

```
var sequence = new Sequence(  
    new Tween<float>(0f, 100f, 1f, EaseType.QuadOut, Lerp, value => x = value),  
    new Tween<float>(100f, 200f, 1f, EaseType.QuadOut, Lerp, value => x = value),  
    new Delay(0.5f),  
    new DoOnce(() => Console.WriteLine("Done!"))  
);  
  
CoroutineManager.Instance.Run(sequence);
```

Concurrent (at the same time):

```
var concurrent = new Concurrent(  
    new Tween<float>(0f, 100f, 1f, EaseType.QuadOut, Lerp, value => x = value),  
    new Tween<float>(0f, 50f, 1.5f, EaseType.SineInOut, Lerp, value => y = value),  
    new Tween<Color>(Color.Red, Color.Blue, 2f, EaseType.QuadInOut, Color.Lerp,  
value => sprite.Color = value)  
);  
  
CoroutineManager.Instance.Run(concurrent);
```

Repeat:

```
// Repeat a tween 5 times  
var repeat = new Repeat(  
    () => new Tween<float>(0f, 100f, 0.5f, EaseType.QuadOut, Lerp, value => x =  
value),  
    count: 5  
);  
  
CoroutineManager.Instance.Run(repeat);  
  
// Repeat indefinitely  
var infiniteRepeat = new Repeat(  
    () => new Tween<float>(0f, 100f, 0.5f, EaseType.QuadOut, Lerp, value => x =  
value),  
    count: -1  
);
```

Conditional Waits

Wait for conditions:

```
// Wait until a flag is true
yield return new WaitUntil(() => isReady);

// Wait while a flag is true
yield return new WaitWhile(() => isAnimating);

// Wait for a value to become non-null
yield return new WaitUntilNotNull<Texture>(() => loadedTexture);

// Wait with a timeout
yield return new WaitUntilOrTimeout(() => isReady, 5f);

// Wait for any of multiple operations
yield return new WaitForAny(
    new WaitUntil(() => isReady),
    new Timeout(new WaitForSeconds(5f), 5f)
);
```

Time Waits

Wait for time:

```
// Wait for 2 seconds (affected by TimeScale)
yield return new WaitForSeconds(2f);

// Wait for 2 real seconds (not affected by TimeScale)
yield return new WaitForSecondsRealtime(2f);

// Wait for 60 frames
yield return new WaitForFrames(60);

// Wait for the next frame
yield return new WaitForNextFrame();

// Wait for a signal
var signal = new WaitForSignal();
yield return signal;
signal.Signal(); // Continues

// Execute every second
CoroutineManager.Instance.Run(new EverySeconds(1f, () =>
    Console.WriteLine("Tick!")));

// Execute every 30 frames
CoroutineManager.Instance.Run(new EveryFrames(30, () =>
    Console.WriteLine("Tick!")));
```

Beacon Waits

Wait for events from the beacon system:

```
// Wait for a beacon
var beaconWait = new WaitForBeacon("PlayerDied");
yield return beaconWait;
var handle = beaconWait.Result;

// Wait with filtering
var waitWithFilter = new WaitForBeacon(
    "DamageEvent",
    h => h.Get<string>(0) == "Player"
);
yield return waitWithFilter;

// Wait for 3 beacons
var waitCount = new WaitForBeaconCount("EnemyKilled", 3);
yield return waitCount;
```

Common Coroutine Patterns

Loading screen:

```
IEnumerator LoadGame()
{
    // Show loading screen
    ShowLoadingScreen();

    // Load assets
    yield return LoadAssetsAsync();

    // Wait for all assets to load
    yield return new WaitUntil(() => allAssetsLoaded);

    // Transition to gameplay
    fsm.ChangeState("Gameplay");
}
```

Boss fight sequence:

```
IEnumerator BossFight()
{
    // Boss enters
    yield return BossEnter();

    // Boss taunts
    yield return BossTaunt();

    // Fight begins
    yield return new WaitUntil(() => bossPhase == BossPhase.Fight);

    // Boss phases
    while (bossHealth > 0)
    {
        yield return BossPhase1();
        if (bossHealth < 75) yield return BossPhase2();
        if (bossHealth < 50) yield return BossPhase3();
        if (bossHealth < 25) yield return BossPhase4();
    }

    // Boss defeated
    yield return BossDefeated();
}
```

Camera shake:

```
IEnumerator CameraShake(float duration, float intensity)
{
    var originalPos = camera.Position;
    float elapsed = 0f;

    while (elapsed < duration)
    {
        elapsed += Time.DeltaTime;
        float offsetX = Random.Range(-intensity, intensity) * (1 - elapsed /
duration);
        float offsetY = Random.Range(-intensity, intensity) * (1 - elapsed /
duration);
        camera.Position = originalPos + new Vect2(offsetX, offsetY);
        yield return null;
    }

    camera.Position = originalPos;
}
```

UI animation sequence:

```
IEnumerator ShowMainMenu()
{
    // Fade in background
    yield return new Tween<float>(0f, 1f, 0.5f, EaseType.QuadOut, Lerp, value =>
bg.Alpha = value);

    // Slide in title (concurrent)
    var titleTween = new Tween<Vect2>(new Vect2(-200, 100), new Vect2(100, 100),
0.5f, EaseType.QuadOut, Vect2.Lerp, value => title.Position = value);

    // Slide in buttons (concurrent)
    var buttonsTween = new Tween<Vect2>(new Vect2(-200, 200), new Vect2(100, 200),
0.5f, EaseType.QuadOut, Vect2.Lerp, value => buttons.Position = value);

    yield return new Concurrent(titleTween, buttonsTween);

    // Wait for player input
    yield return new WaitUntil(() =>
InputAction.GetState().IsPressed("StartGame"));

    // Fade out
    yield return new Tween<float>(1f, 0f, 0.3f, EaseType.QuadIn, Lerp, value =>
bg.Alpha = value);

    fsm.ChangeState("Gameplay");
}
```

Performance Considerations

1. **Avoid creating garbage:** Coroutines allocate `IEnumerator` objects. Pool them if you're creating many.
2. **Don't run too many coroutines at once:** Each coroutine has overhead. If you have hundreds, consider batching.
3. **Use `WaitForSeconds` for time-based waits:** It's more efficient than waiting in a loop with a timer.
4. **Stop coroutines when they're not needed:** Use `CoroutineManager.Instance.Stop(handle)` to stop coroutines.
5. **Use concurrent for parallel tasks:** It's more efficient than running multiple separate coroutines.

What's Next?

- [State Machine](#) — Manage game states
- [Beacons](#) — Decoupled communication
- [Logging](#) — Record what your game is doing

Next: [14. State Machine](#)

14. State Machine

Managing game states with coroutines.

What Is a State Machine?

A state machine manages different states (Idle, Walk, Jump, etc.) and transitions between them. Each state is a coroutine.

Why You Need This

Games have different states. A character can be Idle, Walking, Jumping, or Attacking. Each state has different behavior and different rules for transitioning to other states.

Without a state machine, you'd use a lot of `if` statements to manage states. This gets messy fast. A state machine keeps your code clean and organized.

Without a State Machine

```
if (isIdle)
{
    // Idle behavior
    if (Input.IsPressed("Jump")) { isJumping = true; isIdle = false; }
}
else if (isWalking)
{
    // Walking behavior
    if (Input.IsPressed("Jump")) { isJumping = true; isWalking = false; }
}
else if (isJumping)
{
    // Jumping behavior
    if (isGrounded) { isJumping = false; isIdle = true; }
}
```

With a State Machine

```
var fsm = new StateMachine();

fsm.AddState("Idle", () => IdleState());
fsm.AddState("Walk", () => WalkState());
fsm.AddState("Jump", () => JumpState());

fsm.ChangeState("Idle");
fsm.Update(frameTime);

IEnumerator IdleState()
{
    while (true)
    {
        yield return null;

        if (InputAction.GetState().IsPressed("Jump"))
            yield return "Jump";

        if (InputAction.GetState().IsHeld("Move"))
            yield return "Walk";
    }
}
```

Basic Example

```
var fsm = new StateMachine();

// Define states
fsm.AddState("Idle", () => IdleState());
fsm.AddState("Walk", () => WalkState());
fsm.AddState("Jump", () => JumpState());

// Optional callbacks
fsm.OnEnter(state => Console.WriteLine($"Entering {state}"));
fsm.OnExit(state => Console.WriteLine($"Exiting {state}"));
fsm.OnChanged((from, to) => Console.WriteLine($"{from} -> {to}"));

// Start
fsm.ChangeState("Idle");

// Update every frame
fsm.Update(frameTime);

// Define a state coroutine
IEnumerator IdleState()
{
    while (true)
    {
        // Wait for the next frame
        yield return null;

        // Check for input
        if (InputAction.GetState().IsPressed("Jump"))
            yield return "Jump"; // Transition to Jump state

        if (InputAction.GetState().IsHeld("Move"))
            yield return "Walk";
    }
}

IEnumerator WalkState()
{
    // Walk for 2 seconds, then go back to Idle
    yield return new WaitForSeconds(2f);
    yield return "Idle";
}
```

State Machine Features

Feature	What It Does
AddState()	Register a state with its coroutine factory
ChangeState()	Switch to a new state (cleanly exits current state)
ForceChangeState()	Switch even if already in that state
GoBack()	Return to the previous state (history)
Restart()	Restart the current state
Pause()	Pause the state machine (stops updates)
Resume()	Resume the state machine
Stop()	Stop the state machine entirely
IsInState()	Check the current state
OnEnter	Called when entering any state
OnExit	Called when exiting any state
OnChanged	Called when changing states

State Callbacks

```
fsm.OnEnter(state =>
{
    Console.WriteLine($"Entering {state}");
    // Could play an enter animation, sound effect, etc.
});

fsm.OnExit(state =>
{
    Console.WriteLine($"Exiting {state}");
    // Could play an exit animation, save state, etc.
});

fsm.OnChanged((from, to) =>
{
    Console.WriteLine($"{from} -> {to}");
    // Could update UI, log transitions, etc.
});
```

History (Go Back)

The state machine tracks history. You can return to the previous state:

```
fsm.ChangeState("Menu");
fsm.ChangeState("Options");
fsm.ChangeState("Graphics");

fsm.GoBack(); // Returns to "Options"
fsm.GoBack(); // Returns to "Menu"
```

Pause and Resume

```
// Pause the state machine (stops updates)
fsm.Pause();

// Do something while paused
// ...

// Resume
fsm.Resume();
```

State Pattern Example: Character Controller

```
public class CharacterController
{
    private StateMachine _fsm;
    private Vect2 _position;
    private float _speed = 200f;
    private float _jumpForce = 500f;
    private bool _isGrounded = true;

    public CharacterController()
    {
        _fsm = new StateMachine();

        _fsm.AddState("Idle", () => IdleState());
        _fsm.AddState("Walk", () => WalkState());
        _fsm.AddState("Jump", () => JumpState());
        _fsm.AddState("Fall", () => FallState());

        _fsm.OnEnter(OnStateEnter);
        _fsm.OnExit(OnStateExit);

        _fsm.ChangeState("Idle");
    }

    public void Update(FrameTime time) => _fsm.Update(time);

    private void OnStateEnter(string state)
    {
        // Play enter animation for this state
    }

    private void OnStateExit(string state)
```

```

{
    // Play exit animation for this state
}

private IEnumerator IdleState()
{
    while (true)
    {
        yield return null;

        if (!_isGrounded)
        {
            yield return "Fall";
        }
        else if (InputAction.GetState().IsPressed("Jump"))
        {
            _position.Y -= _jumpForce * Time.deltaTime;
            _isGrounded = false;
            yield return "Jump";
        }
        else if (InputAction.GetState().IsHeld("Move"))
        {
            yield return "Walk";
        }
    }
}

private IEnumerator WalkState()
{
    while (true)
    {
        yield return null;

        if (!_isGrounded)
        {
            yield return "Fall";
        }
        else if (InputAction.GetState().IsPressed("Jump"))
        {
            _position.Y -= _jumpForce * Time.deltaTime;
            _isGrounded = false;
            yield return "Jump";
        }
        else if (!InputAction.GetState().IsHeld("Move"))
        {
            yield return "Idle";
        }

        // Move
        var direction = InputAction.GetState().GetState("Move");
        _position += direction * _speed * Time.deltaTime;
    }
}

private IEnumerator JumpState()
{
    // Jump animation
}

```

```

        float timer = 0f;

        while (timer < 0.5f)
        {
            yield return null;
            timer += Time.deltaTime;

            if (_isGrounded)
                break;
        }

        yield return "Fall";
    }

    private IEnumerator FallState()
    {
        while (true)
        {
            yield return null;

            if (_isGrounded)
                yield return "Idle";
        }
    }
}

```

State Pattern Example: Game States

```

public class GameManager
{
    private StateMachine _fsm;

    public void Initialize()
    {
        _fsm = new StateMachine();

        _fsm.AddState("Splash", () => SplashState());
        _fsm.AddState("MainMenu", () => MainMenuState());
        _fsm.AddState("Loading", () => LoadingState());
        _fsm.AddState("Gameplay", () => GameplayState());
        _fsm.AddState("Paused", () => PausedState());
        _fsm.AddState("GameOver", () => GameOverState());

        _fsm.ChangeState("Splash");
    }

    public void Update(FrameTime time) => _fsm.Update(time);

    private IEnumerator SplashState()
    {
        ShowSplashScreen();
        yield return new WaitForSeconds(3f);
        HideSplashScreen();
    }
}

```

```

        yield return "MainMenu";
    }

    private IEnumerator MainMenuState()
    {
        ShowMainMenu();

        while (true)
        {
            yield return null;

            if (InputAction.GetState().IsPressed("StartGame"))
            {
                HideMainMenu();
                yield return "Loading";
            }

            if (InputAction.GetState().IsPressed("Quit"))
                QuitGame();
        }
    }

    private IEnumerator LoadingState()
    {
        ShowLoadingScreen();

        yield return LoadAssetsAsync();
        yield return new WaitUntil(() => allAssetsLoaded);

        HideLoadingScreen();
        yield return "Gameplay";
    }

    private IEnumerator GameplayState()
    {
        while (true)
        {
            yield return null;

            if (InputAction.GetState().IsPressed("Pause"))
                yield return "Paused";

            if (playerHealth <= 0)
                yield return "GameOver";
        }
    }

    private IEnumerator PausedState()
    {
        ShowPauseMenu();

        while (true)
        {
            yield return null;

            if (InputAction.GetState().IsPressed("Pause"))
            {

```

```

        HidePauseMenu();
        yield return "Gameplay";
    }

    if (InputAction.GetState().IsPressed("Quit"))
    {
        HidePauseMenu();
        yield return "MainMenu";
    }
}

private IEnumerator GameOverState()
{
    ShowGameOverScreen();

    while (true)
    {
        yield return null;

        if (InputAction.GetState().IsPressed("Restart"))
        {
            HideGameOverScreen();
            yield return "Loading";
        }

        if (InputAction.GetState().IsPressed("Quit"))
        {
            HideGameOverScreen();
            yield return "MainMenu";
        }
    }
}
}

```

Common State Machine Patterns

State with Coroutine Sub-states:

```

IEnumerator AttackState()
{
    // Start attack
    yield return AttackWindup();

    // Attack active
    yield return AttackActive();

    // Attack cooldown
    yield return AttackCooldown();

    // Return to idle
    yield return "Idle";
}

IEnumerator AttackWindup()
{

```

```

    _animation.Play("AttackWindup");
    yield return new WaitForSeconds(0.2f);
}

IEnumerator AttackActive()
{
    _animation.Play("AttackActive");
    CheckHitbox();
    yield return new WaitForSeconds(0.1f);
}

IEnumerator AttackCooldown()
{
    _animation.Play("AttackCooldown");
    yield return new WaitForSeconds(0.3f);
}

```

State with Conditional Transitions:

```

IEnumerator EnemyState()
{
    while (true)
    {
        yield return null;

        // Check distance to player
        float distance = Vect2.Distance(_position, _playerPosition);

        if (distance < 50)
            yield return "Melee";
        else if (distance < 200)
            yield return "Ranged";
        else
            yield return "Patrol";
    }
}

```

What's Next?

- [Beacons](#) — Decoupled communication
- [Logging](#) — Record what your game is doing
- [Helpers](#) — Utility functions

Next: [15. Beacons \(Pub/Sub\)](#)

15. Beacons (Pub/Sub)

Decoupled communication between systems.

What Are Beacons?

Beacons are a pub/sub (publish/subscribe) system. They let different parts of your game communicate without knowing about each other directly.

Why You Need This

Without beacons, systems need direct references to each other:

```
// The player needs to tell the UI to update
player.OnHealthChanged += ui.UpdateHealthBar;
player.OnDied += ui.ShowGameOver;

// The player needs to tell the achievement system
player.OnDied += achievementSystem.OnPlayerDied;

// This creates tight coupling
```

This makes your code hard to change and hard to test. If you add a new system that needs to know about player deaths, you have to modify the player class.

Beacons solve this. Systems publish events and other systems subscribe to them. The publisher doesn't know who's listening. The subscriber doesn't know who published.

Basic Example

```
// Subscribe to a topic
BeaconManager.Instance.Subscribe("PlayerDied", handle =>
{
    var position = handle.Get<Vect2>(0);
    var health = handle.Get<int>(1);
    Console.WriteLine($"Player died at {position} with health {health}");
});

// Publish a beacon
BeaconManager.Instance.Publish("PlayerDied", playerPosition, playerHealth);

// Unsubscribe
BeaconManager.Instance.Unsubscribe("PlayerDied", handler);
```

Getting Data from Beacons

```
BeaconManager.Instance.Subscribe("DamageEvent", handle =>
{
    // Get data by index
    string target = handle.Get<string>(0);
    int damage = handle.Get<int>(1);
    Vect2 position = handle.Get<Vect2>(2);

    // Safe access
    if (handle.TryGet<string>(0, out var target))
    {
        // target is valid
    }
});
```

Why Use Beacons?

Benefit	What It Means
Decoupling	Systems don't need to know about each other
Flexibility	Add new subscribers without changing the publisher
Testing	Easier to test isolated systems
Modularity	Systems can be added or removed independently

Real-World Example

Without Beacons (Tight Coupling):

```
public class Player
{
    private UI _ui;
    private AchievementSystem _achievements;
    private SoundSystem _sounds;
    private SaveSystem _save;

    public void TakeDamage(int damage)
    {
        Health -= damage;
        _ui.UpdateHealthBar(Health); // Direct reference
        _sounds.Play("hurt");        // Direct reference

        if (Health <= 0)
        {
            _ui.ShowGameOver();        // Direct reference
            _achievements.OnPlayerDied(); // Direct reference
            _save.Save();              // Direct reference
        }
    }
}
```

With Beacons (Decoupled):

```
public class Player
{
    public void TakeDamage(int damage)
    {
        Health -= damage;

        // Publish that damage was taken
        BeaconManager.Instance.Publish("PlayerDamaged", Health, damage);

        if (Health <= 0)
        {
            // Publish that the player died
            BeaconManager.Instance.Publish("PlayerDied", Position, Health);
        }
    }
}

// UI system subscribes
BeaconManager.Instance.Subscribe("PlayerDamaged", handle =>
{
    int health = handle.Get<int>(0);
    _ui.UpdateHealthBar(health);
});

BeaconManager.Instance.Subscribe("PlayerDied", handle =>
{
    _ui.ShowGameOver();
});

// Achievement system subscribes
BeaconManager.Instance.Subscribe("PlayerDied", handle =>
{
    _achievements.OnPlayerDied();
});

// Sound system subscribes
BeaconManager.Instance.Subscribe("PlayerDamaged", handle =>
{
    _sounds.Play("hurt");
});

// Save system subscribes
BeaconManager.Instance.Subscribe("PlayerDied", handle =>
{
    _save.Save();
});
```

Using Enums for Topics

```
public enum GameEvents
{
    PlayerDied,
    PlayerDamaged,
    EnemyKilled,
    ScoreChanged,
    LevelComplete
}

// Subscribe
BeaconManager.Instance.Subscribe(GameEvents.PlayerDied, handle =>
{
    Console.WriteLine("Player died!");
});

// Publish
BeaconManager.Instance.Publish(GameEvents.PlayerDied, playerPosition,
playerHealth);
```

Advanced Usage

Multiple subscribers:

```
// Multiple systems can subscribe to the same event
BeaconManager.Instance.Subscribe("ScoreChanged", UpdateScoreUI);
BeaconManager.Instance.Subscribe("ScoreChanged", CheckAchievements);
BeaconManager.Instance.Subscribe("ScoreChanged", SaveScore);
```

Filtering:

```
// Subscribe with a predicate
BeaconManager.Instance.Subscribe("DamageEvent", handle =>
{
    // Only handle if the damage is from fire
    string damageType = handle.Get<string>(2);
    if (damageType == "Fire")
    {
        // Handle fire damage
    }
});

// Or use a predicate when subscribing (WaitForBeacon)
var beaconWait = new WaitForBeacon(
    "DamageEvent",
    h => h.Get<string>(2) == "Fire"
);
yield return beaconWait;
```

Coroutine-based waiting:

```
// Wait for a beacon in a coroutine
IEnumerator WaitForPlayerDeath()
{
    var beaconWait = new WaitForBeacon("PlayerDied");
    yield return beaconWait;

    Console.WriteLine("Player died!");
    var handle = beaconWait.Result;
    var position = handle.Get<Vect2>(0);
}
```

When to Use Beacons vs Direct Calls

Use Beacons When	Use Direct Calls When
Multiple systems need to know about an event	One system needs to call another
You don't know who needs to know	You know exactly who needs to know
You want to keep systems decoupled	Systems are already tightly coupled
The event is global or cross-system	The call is local to one system
You want to add new listeners without changing the publisher	You're not going to add more listeners

What's Next?

- [Logging](#) — Record what your game is doing
- [Helpers](#) — Utility functions
- [Extensions](#) — Adding new powers to existing types

Next: [16. Logging](#)

16. Logging

Recording what your game is doing.

What Is Logging?

Logging is recording what your game is doing. It's useful for debugging, finding bugs, and understanding what happened when something went wrong.

Why You Need This

Games are complex. When something goes wrong, you need to know what happened. Without logging, you're guessing. With logging, you have a record of what your game was doing when it crashed.

Basic Example

```
// Get the logger
var logger = Logger.Instance;

// Log messages
logger.Info("Game started");
logger.Warning("Low memory warning");
logger.Error("Failed to load texture: player.png");
logger.Fatal("Critical error, shutting down");

// Log with categories
logger.InfoWithCategory("Network", "Connected to server");
logger.WarningWithCategory("Audio", "Sound pool is full");
logger.ErrorWithCategory("Assets", "Failed to load texture");
```

Log Levels

Level	What It Means	When To Use
Debug	Detailed information for developers	Development and troubleshooting
Info	Normal operation messages	General game events
Warning	Something unusual but not fatal	Potentially problematic situations
Error	Something went wrong but the game continues	Recoverable failures
Fatal	The game is going to crash	Unrecoverable failures

Log Sinks

Sinks are where log messages go.

ConsoleSink: Writes to the console with colors.

```
Logger.Instance.AddSink(new ConsoleSink());
```

FileSink: Writes to daily rotating files.

```
Logger.Instance.AddSink(new FileSink(  
    logFolder: "Logs/",  
    maxFileSizeMB: 10,  
    maxFiles: 10  
));
```

Custom Log Sink

```
public class DatabaseSink : ILogSink  
{  
    public void Write(LogEntry entry)  
    {  
        // Send log entry to a database  
    }  
}  
  
Logger.Instance.AddSink(new DatabaseSink());
```

Categories

Categories help you organize logs by system:

```
Logger.Instance.InfoWithCategory("Physics", "Player collided with wall");  
Logger.Instance.WarningWithCategory("Audio", "Failed to load sound file");  
Logger.Instance.ErrorWithCategory("Network", "Connection timeout");
```

Async Logging

Void's logging is asynchronous. This means logging doesn't block your game.

When you call `logger.Info()`, the message is queued and processed in the background. Your game keeps running without waiting for the log to be written.

What's Next?

- [Helpers](#) — Utility functions
- [Extensions](#) — Adding new powers to existing types
- [Performance](#) — Making your game run fast

Next: [17. Helpers](#)

17. Helpers

Utility functions for common tasks.

What Are Helpers?

Helpers are utility functions that make common tasks easier. They're used throughout the engine and available for your game.

Why You Need This

Many tasks in game development are repetitive. You need to:

- Position UI elements
- Detect collisions
- Work with files
- Create hashes
- Do math calculations

Helpers give you simple functions for all of these tasks. You don't have to write them yourself.

AlignHelpers

Position UI elements.

```
// Center something on screen
Vect2 center = AlignHelpers.AlignCenter(new Vect2(1920, 1080), new Vect2(100, 50));

// Align to the top-right with offset
Vect2 pos = AlignHelpers.AlignToContainer(
    containerSize: new Vect2(1920, 1080),
    childSize: new Vect2(100, 50),
    hAlign: HAlign.Right,
    vAlign: VAlign.Top,
    offset: new Vect2(-10, 10)
);

// Distribute items evenly
float[] positions = AlignHelpers.DistributeEvenly(
    containerSize: 1000,
    itemCount: 5,
    itemSize: 50,
    spacing: 20
);
```

HAlign Values:

Value	What It Does
Left	Align to the left edge
Center	Center horizontally
Right	Align to the right edge
Stretch	Fill the available width

VAlign Values:

Value	What It Does
Top	Align to the top edge
Center	Center vertically
Bottom	Align to the bottom edge
Stretch	Fill the available height

CollisionHelper

Detect collisions between shapes and resolve overlaps.

```
// Check if two rectangles overlap
if (CollisionHelper.RectRect(playerRect, wallRect))
{
    // Handle collision
}

// Check if a rectangle and circle overlap
if (CollisionHelper.RectCircle(wallRect, circleCenter, circleRadius))
{
    // Handle collision
}

// Check if a point is inside a rectangle
if (CollisionHelper.PointRect(mousePosition, buttonRect))
{
    // Mouse is over the button
}

// Resolve an overlap
Vect2 push = CollisionHelper.PushRectRect(playerRect, wallRect);
playerRect.Position += push;

// Reflect a velocity off a surface
Vect2 newVelocity = CollisionHelper.Reflect(velocity, normal, bounciness: 0.8f);

// Move and slide along walls
Vect2 newPosition = CollisionHelper.MoveAndSlideRect(
    playerRect,
    velocity,
    obstacles,
    iterations: 4
);

// Raycast against rectangles and circles
if (CollisionHelper.RaycastRect(origin, direction, rect, out var hitPoint, out var distance))
{
    // Hit at hitPoint
}

// Distance between shapes
float distance = CollisionHelper.DistanceRectRect(rect1, rect2);
```

FileHelper

Work with files and folders.

```
// Get the application data folder
string appData = FileHelper.GetApplicationData("MyStudio", "MyGame");

// Normalize a path (converts backslashes to forward slashes, removes ..)
string normalized = FileHelper.Normalize("folder\\subfolder/../file.txt");
// Returns: "folder/file.txt"

// Ensure a directory exists
FileHelper.EnsureDirectoryExists("Saves/");

// Check if a file is in use
if (!FileHelper.IsFileInUse("myfile.txt"))
{
    // File is safe to read or write
}
```

HashHelper

Create FNV-1a hashes (fast, non-cryptographic).

```
// Hash a string
uint hash32 = HashHelper.Hash32("Hello World");
ulong hash64 = HashHelper.Hash64("Hello World");

// Hash with caching (faster for repeated hashing)
uint cached32 = HashHelper.Cache32("Hello World");
ulong cached64 = HashHelper.Cache64("Hello World");

// Hash an enum
uint enumHash = HashHelper.Cache32(MyEnum.Value);
```

MathHelper

General math utilities.

```
// Interpolation
float value = MathHelper.Lerp(0f, 10f, 0.5f); // 5f
float smooth = MathHelper.SmoothStep(0f, 10f, 0.5f);

// Clamping
float clamped = MathHelper.Clamp(value, 0f, 1f);

// Wrapping
float wrapped = MathHelper.Wrap(angle, -PI, PI);

// Conversion
float radians = MathHelper.ToRadians(90f);
float degrees = MathHelper.ToDegrees(PI);

// Snapping
float snapped = MathHelper.Snap(12.3f, 5f); // 10f

// Remapping
float remapped = MathHelper.Remap(50f, 0f, 100f, 0f, 1f); // 0.5f
```

MapHelper

Tile ↔ world coordinate conversion and spatial queries.

```
// Convert tile to world
Vect2 worldPos = MapHelper.MapToWorld(new Vect2(5, 3), tileSize: 32);

// Convert world to tile
Vect2 tilePos = MapHelper.WorldToMap(new Vect2(160, 96), tileSize: 32);

// 1D to 2D index
Vect2 tile2D = MapHelper.To2D(index, mapWidth: 20);
int index = MapHelper.To1D(tile2D, mapWidth: 20);

// Get tiles in a circle
var tiles = MapHelper.ToCircle(center, radius: 3);

// Get tiles along a line
var lineTiles = MapHelper.ToLine(start, end);

// Flood fill
var walkableTiles = MapHelper.FloodFill(start, tile => IsWalkable(tile));

// Check if two tiles are adjacent
bool adjacent = MapHelper.IsUnitAround(pos1, pos2, includeCorners: true);
```

SoundHelper

Audio utilities.

```
// Master volume
SoundHelper.MasterVolume = 0.7f;

// Category volumes
SoundHelper.SetCategoryVolume(SoundCategory.Music, 0.5f);
SoundHelper.SetCategoryVolume(SoundCategory.SFX, 0.8f);

// Batch operations
SoundHelper.StopAll();
SoundHelper.PauseAll();
SoundHelper.ResumeAll();

// Sound groups
SoundHelper.RegisterSoundGroup("footsteps", footstep1, footstep2, footstep3);
SoundHelper.PlayFromGroup("footsteps", volume: 0.7f);

// Random utilities
float pitch = SoundHelper.RandomPitch(pitchRange: 0.15f);
float pan = SoundHelper.RandomPan(panRange: 1f);

// Pool stats
int active = SoundHelper.ActiveSoundCount;
int available = SoundHelper.AvailableSoundCount;
bool exhausted = SoundHelper.IsPoolExhausted;
```

TextHelper

Text manipulation.

```
// Wrap text to fit a width
string wrapped = TextHelper.WrapText(font, longText, maxWidth: 200f);

// Truncate with ellipsis
string truncated = TextHelper.TruncateWithEllipsis(font, text, maxWidth: 100f);

// Ellipsize in the middle (good for file paths)
string path = TextHelper.EllipsizeMiddle(font,
"C:/Users/Username/Documents/file.txt", maxWidth: 150f);

// Format time
string time = TextHelper.FormatTime(125.5f); // "2:05"
string longTime = TextHelper.FormatTimeLong(3665f); // "1:01:05"

// Clean text (remove extra whitespace)
string cleaned = TextHelper.CleanText(text);

// Measure text
float wrappedHeight = TextHelper.MeasureWrappedHeight(font, text, maxWidth: 200f);
int lineCount = TextHelper.CountLines(text);
float widestLine = TextHelper.GetWidestLine(font, text);
```

DiscoverableHelper

Reflection-based type discovery.

```
// Mark a class as discoverable
[Discoverable(Name = "MyMod", Category = "Gameplay", Priority = 10)]
public class MyMod : IMod { }

// Find all discoverable types implementing IMod
var mods = DiscoverableHelper.FindAll<IMod>();

// Find a specific mod by name
var mod = DiscoverableHelper.FindSingleByName<IMod>("MyMod");

// Find mods by category
var gameplayMods = DiscoverableHelper.FindManyByCategory<IMod>("Gameplay");

// Find mods by name and category
var specificMod = DiscoverableHelper.FindSingleByNameAndCategory<IMod>("MyMod",
"Gameplay");
```

InstanceHelper

Reflection-based object creation.

```
// Create an instance by name
var instance = InstanceHelper.CreateInstance<IMod>("MyMod", ignoreCase: true, args:
null);

// Try create with out parameter
if (InstanceHelper.TryCreateInstance<IMod>("MyMod", true, null, out var mod))
{
    mod.Initialize();
}

// Create from type reference
var instance = InstanceHelper.CreateInstanceFromType<IMod>(type, null);

// Refresh assemblies after loading new mods
InstanceHelper.RefreshAssemblies();
```

JsonHelper

JSON parsing utilities.

```
using var doc = JsonDocument.Parse(jsonString);
var root = doc.RootElement;

// Get a property with default value
string name = root.GetPropertyOrDefault("name", "default");
int age = root.GetPropertyOrDefault("age", 0);
bool enabled = root.GetPropertyOrDefault("enabled", true);

// Get an element directly
string value = root.GetElementOrDefault("hello");
```

What's Next?

- [Extensions](#) — Adding new powers to existing types
- [Performance](#) — Making your game run fast
- [Content Pipeline](#) — How assets flow from file to screen

Next: [18. Extensions](#)

18. Extensions

Adding new powers to existing types.

What Are Extensions?

Extensions add new methods to existing types. They make your code cleaner, more readable, and more natural to write.

Why You Need This

When you're writing game code, you often wish that a type like `float` or `string` had an extra method. Maybe you want to write `myFloat.Clamp(0, 1)` instead of `MathHelper.Clamp(myFloat, 0, 1)`. Or maybe you want to write `myString.IsEmpty()` instead of `string.IsNullOrEmpty(myString)`.

Extensions let you do exactly that. They add new methods to existing types without changing the original code.

How It Works

Extensions are static methods that look like instance methods. They take the type they're extending as the first parameter with the `this` keyword.

Float Extensions

```
float value = 1.5f;

value = value.Clamp(0f, 1f);      // 1f
value = value.Saturate();         // 1f
float wrapped = 5f.Wrap(0f, 3f);  // 2f
float radians = 90f.ToRadians();  // PI/2
float degrees = PI.ToDegrees();   // 180f

int rounded = 3.7f.RoundToInt();  // 4
int floored = 3.7f.FloorToInt();  // 3
int ceiled = 3.2f.CeilToInt();    // 4

float snapped = 12.3f.Snap(5f);   // 10f
string percent = 0.75f.ToPercent(); // "75%"
string time = 125f.ToTimeString(); // "2:05"
float lerped = 0f.LerpTo(10f, 0.5f); // 5f

bool isZero = 0.001f.IsZero();    // false (uses epsilon)
bool isEqual = 0.1f.ApproxEquals(0.1001f); // true
```

Int Extensions

```
int value = 5;

value = value.Clamp(0, 3);           // 3
int wrapped = 7.Wrap(0, 5);          // 2
bool isEven = 4.IsEven();            // true
bool isOdd = 3.IsOdd();              // true
bool isPower = 8.IsPowerOfTwo();     // true
int nextPower = 5.NextPowerOfTwo();  // 8

// Time conversion
int ms = 2.SecondsToMilliseconds(); // 2000
float seconds = 1500.MillisecondsToSeconds(); // 1.5f

// Frame conversion
float secs = 120.FramesToSeconds(60); // 2.0f
int frames = 3.SecondsToFrames(60); // 180

// Tile conversion
int tileX = 128.PixelsToTiles(32);   // 4
int pixelX = 5.TilesToPixels(32);    // 160

bool isTrue = 1.ToBool();             // true
int abs = (-5).Abs();                 // 5
int sign = (-5).Sign();               // -1
```

String Extensions

```
string text = "Hello World";

// Empty checks
bool empty = text.IsEmpty(); // false
bool notEmpty = text.IsNotEmpty(); // true

// Numeric checks
bool isInt = "123".IsInteger(); // true
bool isDecimal = "12.34".IsDecimal(); // true
bool isNumeric = "12.34".IsNumeric(); // true

// Trimming
string trimmed = "Hello World".TrimToLength(5); // "Hello"

// Counting
int count = "Hello World".CountChar('l'); // 3
int subCount = "Hello Hello".CountSubstring("Hello"); // 2

// Pattern matching
bool starts = "Hello World".StartsWithAny("He", "Wo"); // true
bool ends = "Hello World".EndsWithAny("rld", "ld"); // true
bool contains = "Hello World".ContainsAny("ell", "xyz"); // true
bool containsAll = "Hello World".ContainsAll("Hello", "World"); // true

// Whitespace removal
string noSpace = "Hello World".RemoveWhitespace(); // "HelloWorld"

// Split and trim
string[] parts = "one, two, three".SplitAndTrim(',');

// Reverse
string reversed = "Hello".Reverse(); // "olleH"

// Take and last
string first = "Hello World".Take(5); // "Hello"
string last = "Hello World".Last(5); // "World"

// Remove from ends
string removedEnd = "Hello World".RemoveEnd(6); // "Hello"
string removedStart = "Hello World".RemoveStart(6); // "World"

// Join collection
string joined = new[] { "a", "b", "c" }.JoinToString(", "); // "a, b, c"
```

IEnumerable Extensions

```
var items = new List<int> { 1, 2, 3, 4, 5 };

// Check if empty
if (items.IsEmpty())
{
    // Iterate
    items.ForEach(x => Console.WriteLine(x));

    // Get random element
    int random = items.Random();

    // Partition
    var (evens, odds) = items.Partition(x => x % 2 == 0);

    // Shuffle
    var shuffled = items.Shuffle();

    // Random sample
    var sample = items.RandomSample(3);

    // Distinct by key
    var distinct = items.DistinctBy(x => x % 2);

    // Check all distinct
    bool allDistinct = items.AllDistinct();
}
```

Enum Extensions

```
[Flags]
public enum MyFlags
{
    None = 0,
    Option1 = 1 << 0,
    Option2 = 1 << 1,
    Option3 = 1 << 2
}

var flags = MyFlags.Option1;

flags = flags.SetFlag(MyFlags.Option2);    // Add a flag
flags = flags.ClearFlag(MyFlags.Option1);  // Remove a flag
flags = flags.ToggleFlag(MyFlags.Option3); // Toggle a flag

bool hasAll = flags.HasAllFlags(MyFlags.Option2 | MyFlags.Option3);
bool hasAny = flags.HasAnyFlag(MyFlags.Option1 | MyFlags.Option2);
bool hasOnly = flags.HasOnlyFlag(MyFlags.Option3);
bool hasNone = flags.HasNoFlags();
```

Random Extensions

```
var rng = FastRandom.Shared;

// Random element
string selected = rng.Choice(new[] { "a", "b", "c" });

// Random direction
Vect2 dir4 = rng.RandomDirection(); // Up, Right, Down, Left
Vect2 dir8 = rng.RandomDirection8Way(); // 8 directions

// Random colors
Color color = rng.RandomColor();
Color pastel = rng.RandomPastelColor();
Color dark = rng.RandomDarkColor();

// Random enum
MyEnum value = rng.RandomEnum<MyEnum>();

// Random sign
int sign = rng.NextSign(); // 1 or -1

// Shuffle a list
rng.Shuffle(myList);

// Dice roll
int sum = rng.RollDice(3, 6); // 3d6

// Random point in circle
Vect2 point = rng.RandomPointInCircle(5f);

// Random angle
float angle = rng.RandomAngle();

// Probability check
if (rng.Chance(0.25f)) // 25% chance
    DoSomething();
```

Sound Extensions

```
// Play a sound one-shot
sound.PlayOneShot(volume: 0.8f, pan: 0f, pitch: 1f, category: SoundCategory.SFX);

// Play and forget (auto-disposes when done)
sound.PlayAndForget(volume: 0.8f);

// Play with pitch variation
sound.PlayWithPitchVariation(pitchRange: 0.15f, volume: 0.8f);

// Fluent configuration
sound.CreateInstance()
    .WithVolume(0.8f)
    .WithPan(-0.5f)
    .WithPitch(1.2f)
    .WithLooping(true)
```

```
.PlayWith(0.8f, -0.5f, 1.2f);

// Stop and dispose
instance.StopAndDispose();

// Play all sounds in a collection
var sounds = new[] { sound1, sound2, sound3 };
sounds.PlayAll(0.7f);
sounds.PlayAllAndForget(0.7f);

// Play random sound from collection
var randomInstance = sounds.PlayRandom(0.8f);
var randomVaried = sounds.PlayRandomWithVariation(0.15f, 0.8f);
```

What's Next?

- [Performance](#) — Making your game run fast
- [Content Pipeline](#) — How assets flow from file to screen
- [Extending Void](#) — Making the engine your own

Next: [19. Performance](#)

19. Performance

Making your game run fast.

What Is Performance?

Performance is how fast your game runs. A game that runs at 60 frames per second feels smooth and responsive. A game that runs at 30 frames per second feels sluggish. A game that stutters is frustrating to play.

Why Performance Matters

Players expect games to run smoothly. A game that stutters or lags is not fun to play. Good performance is essential for a good player experience.

How Void Helps

Void includes many performance optimizations built in:

Feature	What It Does
Texture atlasing	Reduces draw calls by packing textures together
Batch rendering	Draws many sprites in one call
Sound pooling	Reuses sound instances
Zero-GC input	No garbage from input queries
LRU asset eviction	Frees memory when not needed
Adaptive compression	Only compresses when beneficial

Performance Targets

- 60 FPS target
- Minimal draw calls
- Low memory usage
- Fast load times

Measuring Performance

FPS Counter:

```
// FrameTime gives you FPS
protected override void OnDraw(FrameTime time)
{
    float fps = time.FPS;
    batcher.DrawText(font, $"FPS: {fps}", new Vect2(10, 10), Color.White);
}
```

Draw Call Count:

```
// BatchStats gives you draw call info
var stats = batcher.Stats;
int drawCalls = stats.DrawCalls;
int vertices = stats.Vertices;
```

Atlas Metrics:

```
var metrics = AtlasManager.Instance.GetMetrics();
float usage = metrics.PercentageFull;
int textureCount = metrics.TextureCount;
int evictions = metrics.EvictionCount;
```

Common Performance Bottlenecks

1. Too many draw calls

Problem: Each draw call has overhead. 100 draw calls is fine. 1000 draw calls is slow.

Solution: Use batching (`SpriteBatcher`). Use texture atlasing (automatic). Combine small textures.

2. Garbage Collection (GC)

Problem: Allocating objects creates garbage. The GC has to clean it up, causing stutters.

Solution: Use pooling. Avoid allocations in `Update/OnDraw`. Use structs instead of classes.

3. Memory usage

Problem: Using too much memory causes slow loading and crashes.

Solution: Use `AssetEviction` to unload unused assets. Use packs for release.

4. CPU usage

Problem: Too much work per frame.

Solution: Spread work across frames with coroutines. Use fixed timestep for physics.

Optimization Tips

Rendering:

```
// Do this: Use SpriteBatcher
var batcher = new SpriteBatcher();
batcher.Begin();
batcher.Draw(...);
batcher.Draw(...);
batcher.Draw(...);
batcher.End();
```

```
// Not this: Draw individually
Draw(texture1, pos1);
Draw(texture2, pos2);
Draw(texture3, pos3);
```

Audio:

```
// Do this: Use sound pooling (automatic)
sound.PlayOneShot(0.8f);

// Not this: Create new instances each time
var instance = new SoundInstance(sound);
instance.Play();
```

Assets:

```
// Do this: Cache assets
Texture playerTexture = AssetManager.Instance.Load<Texture>("player.png");

// Not this: Load every frame
Texture playerTexture = AssetManager.Instance.Load<Texture>("player.png");
```

Scripting:

```
// Do this: Cache delegates
var action = InputAction.GetState();
if (action.IsPressed("Jump"))

// Not this: Get state every time
if (InputAction.GetState().IsPressed("Jump"))
```

What's Next?

- [Content Pipeline](#) — How assets flow from file to screen
- [Extending Void](#) — Making the engine your own
- [Debugging & Development Tools](#) — Tools for development

Next: [20. Content Pipeline](#)

20. Content Pipeline

How assets flow from file to screen.

What Is the Content Pipeline?

The content pipeline is the system that takes your raw assets (PNG, WAV, etc.) and prepares them for use in your game. It handles:

1. **Importing** — Reading the raw file format
2. **Processing** — Converting to engine format (atlas packing, compression)
3. **Loading** — Getting the asset into memory at runtime

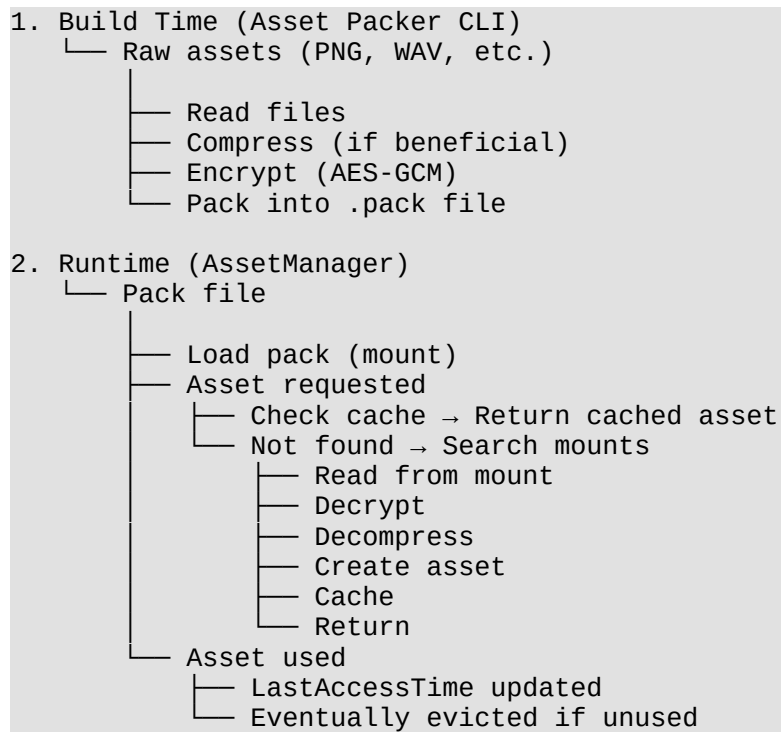
Why You Need This

Without a content pipeline, you'd have to:

- Load files from disk manually
- Parse file formats yourself
- Manage memory manually
- Handle asset dependencies

Void's content pipeline handles all of this for you.

The Asset Lifecycle



Built-in Asset Types

Void supports these asset types out of the box:

Asset Type	Load Method	File Extensions
Texture	Load<Texture>()	.png, .bmp, .tga, .jpg, .gif
Sound	Load<Sound>()	.wav, .ogg, .mp3, .flac
Shader	Load<Shader>()	.shader
SpriteFont	Load<SpriteFont>()	.png
Spritesheet	Load<Spritesheet>()	.sheet, .json
LDtkMap	Load<LDtkMap>()	.ldtk, .json

How Assets Are Found

When you call `AssetManager.Load<Texture>("player.png")`:

1. The `AssetManager` normalizes the path (`player.png` → `player.png`)
2. It checks the cache for the asset
3. If not found, it searches mounts in order:
 - `VirtualFileSystemMount` → checks `Content/player.png`
 - Other mounts → checked in order of priority
4. The first mount that has the file reads it
5. The data is loaded and processed
6. The asset is cached and returned

Custom Asset Types

You can add your own asset types by implementing `IAAsset`.

Step 1: Implement `IAAsset`

```
public class MyCustomAsset : IAAsset
{
    public uint Id { get; }
    public string Tag { get; }
    public byte[] Data { get; }
    public bool IsValid { get; private set; }
    public AssetType Type => AssetType.Normal;
    public DateTime LastAccessTime { get; set; }

    public MyCustomAsset(uint id, byte[] data, string tag)
    {
        Id = id;
        Data = data;
        Tag = tag;
    }

    public void Load()
    {
        // Parse the data into your asset
        // For example: load a custom file format
        IsValid = true;
    }

    public void Unload()
    {
        // Free resources
        IsValid = false;
    }

    public void Dispose()
    {
        // Clean up
        IsValid = false;
    }
}
```

Step 2: Register the Asset Type

```
// In your game initialization
AssetManager.Instance.RegisterAssetType<MyCustomAsset>(
    extensions: new[] { ".myasset", ".custom" },
    factory: (id, data, tag) => new MyCustomAsset(id, data, tag)
);
```

Step 3: Load Your Custom Asset

```
MyCustomAsset myAsset =
    AssetManager.Instance.Load<MyCustomAsset>("myfile.myasset");
```

The Mount System

Mounts provide the files that the AssetManager loads. Each mount is a virtual file system.

Built-in Mounts:

Mount	What It Does
VirtualFileSystemMount	Loads from your Content folder
MacOSMount	Loads from macOS app bundles
PackMount	Loads from encrypted pack files

Creating a Custom Mount:

```
public class NetworkMount : IMount
{
    public string Name => "Network Assets";

    public bool HasFile(string virtualPath)
    {
        return NetworkAssetExists(virtualPath);
    }

    public byte[] ReadFile(string virtualPath)
    {
        return NetworkDownload(virtualPath);
    }
}
```

The Pack Integration

Packs are just another type of mount. They provide encrypted, compressed access to assets.

```
// Load a pack
var pack = AssetManager.Instance.LoadPack("GameAssets.pack");

// Add it to the search order
AssetManager.Instance.AddMountToStart(pack);

// Now all assets in the pack are available
Texture playerTexture = AssetManager.Instance.Load<Texture>("player.png");
```

What's Next?

- [Extending Void](#) — Making the engine your own
- [Debugging & Development Tools](#) — Tools for development
- [FAQ](#) — Common questions

Next: [21. Extending Void](#)

21. Extending Void

Making the engine your own.

What Is Extending?

Extending is how you change or add to the engine's behavior. Instead of modifying the engine code, you implement interfaces and register them. This keeps your code separate from the engine and makes upgrades easier.

Why You Might Want to Extend

Void gives you the essentials, but every game is different. You might want to:

- Use a different texture packing algorithm
- Load assets from a network drive or cloud storage
- Send logs to a database instead of a file
- Save custom data types
- Add custom blend modes

Extending lets you do all of these without changing the engine code.

What Can You Extend?

System	Interface	What You Can Do
Atlas Packing	IAtlasPacker	Custom texture packing algorithm
Asset Mounts	IMount	Custom asset source
Log Sinks	ILogSink	Custom log destination
Save/Load	ContentTypeWriterReader<T>	Custom save data types
Blend Modes	IBlendMode	Custom rendering blend modes
Render Targets	IRenderTarget	Custom render targets
Vertex Buffers	IVertexBuffer	Custom vertex buffer
Shaders	IShader	Custom shader implementation

How to Extend

1. **Implement the interface** for the system you want to extend
2. **Register it** with the appropriate manager
3. **The engine finds it** and uses it automatically

Examples

Custom Atlas Packer:

```
public class MyPacker : IAtlasPacker
{
    public int UsedSpace { get; private set; }
    public int TotalSpace { get; private set; }
    public float Fragmentation { get; private set; }

    public MyPacker(int width, int height)
    {
        TotalSpace = width * height;
    }

    public bool TryPack(int width, int height, out Rect2 packedRect)
    {
        // Your custom packing algorithm
        packedRect = default;
        return false;
    }

    public void Clear()
    {
        // Reset the packer
    }

    public List<(Rect2 OldRect, Rect2 NewRect)> Defrag()
    {
        // Defragment the atlas
        return new List<(Rect2, Rect2)>();
    }

    public void Free(Rect2 rect)
    {
        // Free a rectangle
    }
}

// Register it
settings.SetAtlasPacker(typeof(MyPacker));
```

Custom Mount:

```
public class NetworkMount : IMount
{
    public string Name => "Network Assets";

    public bool HasFile(string virtualPath)
    {
        // Check if the file exists on the network
        return NetworkAssetExists(virtualPath);
    }

    public byte[] ReadFile(string virtualPath)
    {
        // Download the file from the network
        return NetworkDownload(virtualPath);
    }
}

AssetManager.Instance.AddMountToStart(new NetworkMount());
```

Custom Log Sink:

```
public class DatabaseSink : ILogSink
{
    public void Write(LogEntry entry)
    {
        // Write to database
        using var connection = new SqlConnection(connectionString);
        connection.Execute("INSERT INTO Logs (Level, Timestamp, Message) VALUES
(@Level, @Timestamp, @Message)",
            new { entry.Level, entry.Timestamp, entry.Message });
    }
}

Logger.Instance.AddSink(new DatabaseSink());
```

Custom Save Data:

```
public class MySaveData
{
    public string Name { get; set; }
    public int Level { get; set; }
}

public class MySaveSystem : ContentTypeWriterReader<MySaveData>
{
    public MySaveSystem(string key) : base(key) { }

    protected override void Write(MySaveData data, ContentWriter writer)
    {
        writer.Write(data.Name);
        writer.Write(data.Level);
    }

    protected override MySaveData Read(ContentReader reader)
    {
        return new MySaveData
        {
            Name = reader.ReadString(),
            Level = reader.ReadInt32()
        };
    }
}
```

Discoverable System

Void includes a reflection-based discovery system for finding and registering types marked with `[Discoverable]`.

```
[Discoverable(Name = "MyMod", Category = "Gameplay", Priority = 10)]
public class MyMod : IMod { }

// Find all discoverable types implementing IMod
var mods = DiscoverableHelper.FindAll<IMod>();

// Find a specific mod by name
var mod = DiscoverableHelper.FindSingleByName<IMod>("MyMod");

// Find mods by category
var gameplayMods = DiscoverableHelper.FindManyByCategory<IMod>("Gameplay");

// Find mods by name and category
var specificMod = DiscoverableHelper.FindSingleByNameAndCategory<IMod>("MyMod",
"Gameplay");
```

Assembly Discovery Settings

```
settings
    .SetDiscoverableScanMode(AssemblyScanMode.ExcludeFramework)
    .AddDiscoverableAssembly("MyModAssembly")
    .AddDiscoverableExcludedPrefix("System.")
    .SetDiscoverableAssemblyFilter(assembly =>
assembly.GetName().Name.StartsWith("My"));
```

AssemblyScanMode:

Mode	What It Does
All	Scan all loaded assemblies
ExcludeFramework	Exclude framework and Void assemblies (default)
Custom	Use custom filter
Whitelist	Only scan explicitly added assemblies
Blacklist	Scan all except explicitly excluded assemblies

What's Next?

- [Debugging & Development Tools](#) — Tools for development
- [FAQ](#) — Common questions

Next: [22. Debugging & Development Tools](#)

22. Debugging & Development Tools

Tools for development.

What Are Debugging Tools?

Debugging tools help you find and fix problems in your game. They show you what's happening inside your game while it's running.

Why You Need This

Games are complex. When something goes wrong, you need to see what's happening. Debugging tools give you visibility into your game's internals.

Built-in Debugging Tools

Void provides several built-in debugging tools:

1. FPS Counter

```
// FrameTime gives you FPS
protected override void OnDraw(FrameTime time)
{
    float fps = time.FPS;
    batcher.DrawText(_font, $"FPS: {fps:F0}", new Vect2(10, 10), Color.White);
}
```

2. Draw Call Counter

```
// BatchStats gives you draw call info
var stats = batcher.Stats;
batcher.DrawText(_font, $"Draw Calls: {stats.DrawCalls}", new Vect2(10, 30),
Color.White);
batcher.DrawText(_font, $"Vertices: {stats.Vertices}", new Vect2(10, 50),
Color.White);
```

3. Atlas Metrics

```
var metrics = AtlasManager.Instance.GetMetrics();
batcher.DrawText(_font, $"Atlas Usage: {metrics.PercentageFull:F1}%", new Vect2(10,
70), Color.White);
batcher.DrawText(_font, $"Textures: {metrics.TextureCount}", new Vect2(10, 90),
Color.White);
batcher.DrawText(_font, $"Pages: {metrics.UsedPages}/{metrics.TotalPages}", new
Vect2(10, 110), Color.White);
```

4. Logging

```
// Log everything
Logger.Instance.Info("Player moved to {0}", playerPosition);
Logger.Instance.Warning("Low health: {0}", health);
Logger.Instance.Error("Failed to load texture: {0}", textureName);
```

5. Debug Overlay Example

```
public class DebugOverlay
{
    private SpriteFont _font;
    private List<string> _debugLines = new();

    public void AddLine(string line)
    {
        _debugLines.Add(line);
        if (_debugLines.Count > 10)
            _debugLines.RemoveAt(0);
    }

    public void Draw(SpriteBatcher batcher)
    {
        float y = 10;
        foreach (var line in _debugLines)
        {
            batcher.DrawText(_font, line, new Vect2(10, y), Color.White);
            y += 20;
        }
    }
}

// Usage
var debug = new DebugOverlay();
debug.AddLine($"FPS: {time.FPS:F0}");
debug.AddLine($"Position: {playerPosition}");
debug.AddLine($"Health: {health}");
debug.Draw(batcher);
```

6. Input Visualizer

```
// Visualize gamepad input
public void DrawInput(GamepadState state, SpriteBatcher batcher)
{
    // Draw stick positions
    Vect2 leftStick = state.LeftStick;
    Vect2 rightStick = state.RightStick;

    // Draw trigger values
    float leftTrigger = state.LeftTrigger;
    float rightTrigger = state.RightTrigger;

    // Draw buttons
    if (state.IsButtonPressed(GamepadButton.A))
        batcher.DrawText(_font, "A", new Vect2(100, 100), Color.Green);
}
```

7. Collision Debug

```
// Draw collision boxes
protected override void OnDraw(FrameTime time)
{
    // Draw player collision box
    batcher.DrawRectOutline(playerRect, Color.Red);

    // Draw enemy collision boxes
    foreach (var enemy in _enemies)
        batcher.DrawRectOutline(enemy.Rect, Color.Yellow);

    // Draw wall collision boxes
    foreach (var wall in _walls)
        batcher.DrawRectOutline(wall.Rect, Color.Blue);
}
```

Performance Profiling

```
public class Profiler
{
    private Dictionary<string, System.Diagnostics.Stopwatch> _timers = new();
    private Dictionary<string, float> _times = new();

    public void Start(string name)
    {
        if (!_timers.ContainsKey(name))
            _timers[name] = new System.Diagnostics.Stopwatch();
        _timers[name].Restart();
    }

    public void Stop(string name)
    {
        _timers[name].Stop();
        _times[name] = (float)_timers[name].Elapsed.TotalMilliseconds;
    }

    public void Draw(SpriteBatcher batcher, SpriteFont font)
    {
        float y = 200;
        foreach (var (name, time) in _times)
        {
            batcher.DrawText(font, $"{name}: {time:F2}ms", new Vect2(10, y),
Color.White);
            y += 20;
        }
    }
}

var profiler = new Profiler();

protected override void OnUpdate(FrameTime time)
{
    profiler.Start("Update");
    // ... game logic ...
    profiler.Stop("Update");
}
```

```
    profiler.Start("AI");  
    // ... AI logic ...  
    profiler.Stop("AI");  
  
    profiler.Start("Physics");  
    // ... physics logic ...  
    profiler.Stop("Physics");  
}
```

What's Next?

- [FAQ](#) — Common questions

Next: [23. FAQ](#)

23. FAQ

Common questions and answers.

General

Q: What is Void? A: Void is a lightweight, extensible 2D game engine. It gives you the essential building blocks every game needs—rendering, audio, input, assets, saving, pathfinding, coroutines, and logging—and then gets out of your way.

Q: Is Void free? A: Yes. Void is open source under the MIT License. You can use it for free for any purpose, including commercial games.

Q: What platforms does Void support? A: Windows, macOS, and Linux.

Q: What programming language does Void use? A: C#. Void targets .NET 8 and above.

Installation

Q: How do I install Void? A: The easiest way is to install the project template: `dotnet new install Void.Templates`. Then create a new game with `dotnet new voidgame -n MyGame`.

Q: Do I need any other dependencies? A: No. Void handles everything through NuGet packages.

Q: Can I use Void with Visual Studio Code? A: Yes. Make sure you have the C# extensions installed.

Development

Q: How do I load an asset? A: Use `AssetManager.Instance.Load<Texture>("player.png")`. The `AssetManager` handles caching and eviction automatically.

Q: How do I play a sound? A: Load the sound with `AssetManager.Instance.Load<Sound>("explosion.wav")`, then play it with `sound.PlayOneShot(0.8f)`.

Q: How do I draw a sprite? A: Create a `SpriteBatcher`, call `Begin()`, draw your sprites with `Draw()`, then call `End()`.

Q: How do I handle input? A: Use `InputAction` to define actions like "Jump", then check them with `InputAction.GetState().IsPressed("Jump")`.

Performance

Q: How do I improve performance? A: Use batching (`SpriteBatcher`), texture atlasing (automatic), sound pooling (automatic), and LRU asset eviction (configure in `GameSettings`).

Q: What's the maximum number of sprites I can draw? A: The batcher handles thousands of sprites efficiently. The exact number depends on your hardware.

Q: How do I reduce draw calls? A: Use texture atlasing (automatic). Combine small textures into larger ones. Use `SpriteBatcher` instead of drawing individually.

Asset Protection

Q: How does Void protect my assets? A: Void packs your assets into encrypted archives using AES-GCM 256-bit encryption. Without the key, the pack is just random bytes.

Q: Can I pack my assets? A: Yes. Use the `packcli` command-line tool to build encrypted packs from your Content folder.

Q: What happens if someone steals the key? A: No system is 100 percent secure. AES-GCM 256-bit makes extraction extremely difficult, but a determined attacker with enough time and resources could find the key. The goal is to stop casual theft.

Extending Void

Q: How do I add custom asset types? A: Implement `IAsset` and register it with `AssetManager.RegisterAssetType()`.

Q: How do I add custom mounts? A: Implement `IMount` and add it with `AssetManager.Instance.AddMountToStart()`.

Q: How do I add custom log sinks? A: Implement `ILogSink` and register it with `Logger.Instance.AddSink()`.

Appendix A: Platform Support

Platform	Status
Windows	Full support
macOS	Full support
Linux	Full support

Appendix B: Dependencies

Dependency	Purpose
.NET 8+	Runtime
SFML	Rendering, windowing, input, audio

Appendix C: License

Void Engine is licensed under the MIT License.

MIT License

Copyright (c) 2026 Void Engine

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Appendix D: File Manifest

File	System
Game.cs	Core
GameSettings.cs	Core
Window.cs	Core
FrameTime.cs	Core
Vect2.cs, Vect3.cs, Vect4.cs	Math
Rect2.cs	Math
Color.cs	Math
Easing.cs	Math
MathHelper.cs, MapHelper.cs, CollisionHelper.cs, AlignHelpers.cs	Helpers
Camera.cs	Rendering
AtlasManager.cs, IAtlasPacker.cs, SkylinePacker.cs, GuillotinePacker.cs, AtlasMetrics.cs	Rendering
IShader.cs, ShaderProgram.cs, ShaderState.cs, Shader.cs	Rendering
IRenderTarget.cs, IVertexBuffer.cs, RenderTarget.cs, TextureRenderTarget.cs, VertexBuffer.cs	Rendering
IBatcher.cs, BaseBatcher.cs, SpriteBatcher.cs, PrimitiveBatcher.cs, BatchStats.cs	Rendering
BlendMode.cs	Rendering
PostProcessor.cs	Rendering
SoundInstancePool.cs, SoundInstance.cs, Sound.cs	Audio
SoundEventArgs.cs, SoundCompletedEventArgs.cs, SoundStoppedEventArgs.cs, SoundLoopedEventArgs.cs, SoundErrorEventArgs.cs	Audio
SoundHelper.cs, SoundExtensions.cs	Audio
Keyboard.cs, KeyboardKeys.cs, KeyboardState.cs, KeyState.cs	Input
Mouse.cs, MouseButton.cs, MouseState.cs	Input
Gamepad.cs, GamepadButton.cs, GamepadState.cs, GamepadDatabase.cs, PlayerIndex.cs, ButtonState.cs	Input
InputAction.cs, InputActionState.cs, ActionBinding.cs, InputBinding.cs	Input

File	System
ContentTypeWriterReader.cs, ContentWriter.cs, ContentReader.cs	Save/Load
AStar2D.cs, FlowField.cs	Pathfinding
CoroutineManager.cs, CoroutineHandle.cs	Coroutines
Tween.cs, CallbackTween.cs, DelayedTween.cs, LoopTween.cs, PingPongTween.cs, PulseTween.cs, SpeedTween.cs	Coroutines
Sequence.cs, Concurrent.cs, Repeat.cs	Coroutines
WaitUntil.cs, WaitUntilNotNull.cs, WaitUntilOrTimeout.cs, WaitWhile.cs, WaitWhileNotNull.cs, WaitWhileOrTimeout.cs, WaitForAny.cs	Coroutines
Delay.cs, EveryFrames.cs, EverySeconds.cs, Timeout.cs, WaitForFrames.cs, WaitForNextFrame.cs, WaitForSeconds.cs, WaitForSecondsRealtime.cs, WaitForSignal.cs	Coroutines
DelayCall.cs, DoOnce.cs	Coroutines
WaitForBeacon.cs, WaitForBeaconCount.cs	Coroutines
StateMachine.cs	FSM
BeaconManager.cs, BeaconHandle.cs	Beacons
AssetManager.cs, IAsset.cs	Assets
Texture.cs, Sound.cs, Shader.cs, SpriteFont.cs, Font.cs, Glyph.cs, Spritesheet.cs, SpritesheetEntry.cs	Assets
IMount.cs, VirtualFileSystemMount.cs, MacOSMount.cs, PackMount.cs, MacOSPackMount.cs	Assets
LDtkMap.cs, LDtkLevel.cs, MapLayer.cs, ILDtkInstance.cs, LDtkEntityInstance.cs, LDtkTileInstance.cs, LDtkIntGridInstance.cs, LDtkTileset.cs, LDtkTile.cs, LDtkSetting.cs, LDtkEntityRef.cs, MapNeighbour.cs	Assets
SolidPackReader.cs, SolidPackBuilder.cs, Packer.cs, PackOptions.cs, PackResult.cs, PackConstants.cs, Exceptions.cs	Packer
AesGcmEncryptor.cs, AdaptiveCompressor.cs, Crc32.cs, PathNormalizer.cs	Packer
Program.cs, BuildCommand.cs, ExtractCommand.cs, VerifyCommand.cs, ListCommand.cs, UpdateCommand.cs, PackService.cs, ResultRenderer.cs, ProgressData.cs, GlobMatcher.cs	CLI

File	System
Logger.cs, LogEntry.cs, ILogSink.cs, ConsoleSink.cs, FileSink.cs	Logging
DiscoverableHelper.cs, FileHelper.cs, HashHelper.cs, InstanceHelper.cs, JsonHelper.cs, TextHelper.cs, EmbeddedResources.cs	Helpers
EnumFlagExtensions.cs, FloatExtensions.cs, FontExtensions.cs, IEnumerableExtensions.cs, IntExtensions.cs, RandExtensions.cs, StringExtensions.cs	Extensions

End of Specification

Void Engine - Build your game, not your engine.

